RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105



AD23B31 Image Processing and Computer Vision
Laboratory Record Note Book

Name : .....

Year / Branch / Section : .....

University Register No. : .....

College Roll No. : .....

Semester : .....

Academic Year : .....

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

AD23B31 Image Processing and Computer Vision

NAME	
ROLL NO	
YEAR	
SEC	

RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: .....

Academic Year: …… **Semester:** …… **Branch:** ……

Register No.

Certified that this is the bonafide record of work done by the above student in the…… Laboratory during the academic year 2025 – 2026.

Signature of Faculty in-charge

Submitted for the Practical Examination held on ……

Internal Examiner

External Examiner

INDEX

SlNO	Experiment	Date	GitHub
1	Installation of Open CV & To perform the basic image handling processing operation on the image	03/01/2026	
2	Implement Edge Detection, Line Detection and Corner Detection	10/01/2026	
3	Demonstrate Camera Calibration using python	24/01/2026	
4	Implement Image Histogram and Histogram Equalization	31/01/2026	
5	Develop a python program for Skin color Detection	07/02/2026	
6	Create a python program for Warping and Estimation	14/02/2026	
7	Develop a python program for Motion Tracking	21/02/2026	
8	Design a program for Object Detection using YOLO	07/03/2026	
9	Develop a python program for Stereo Vision and Depth Estimation	14/03/2026	
10	Demonstrate Augmented reality using feature matching	04/04/2026	
11	Mini Project <project title>	11/04/2026, 18/04/2026, 25/04/2026, 02/05/2026	

Name: S SHUVANTH Register Number: 250701708
Experiment Name:Random Sampling and Sampling Distribution

```
import numpy as np
import matplotlib.pyplot as plt

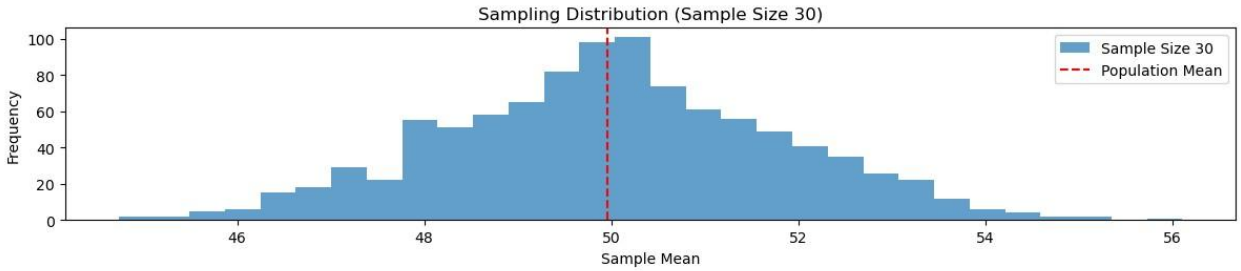
# Step 1: Generate a population (e.g., normal distribution)
population_mean = 50 population_std = 10 population_size =
100000
population = np.random.normal(population_mean, population_std,
population_size)

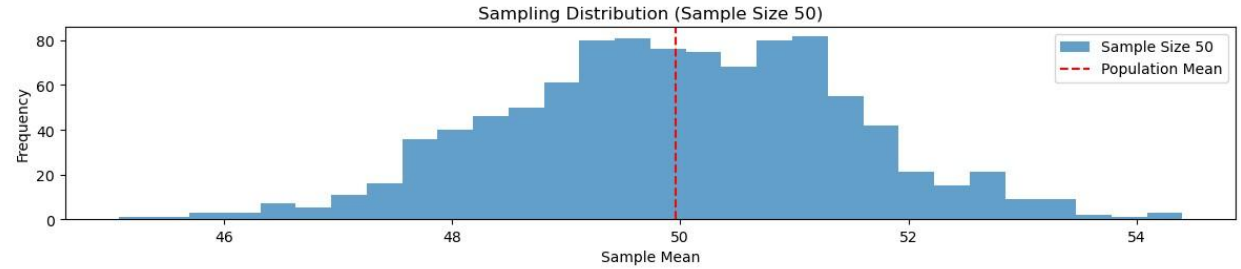
# Step 2: Random sampling
sample_sizes = [30, 50, 100] # different sample sizes to consider
num_samples = 1000 # number of samples for each sample size
sample_means = {}

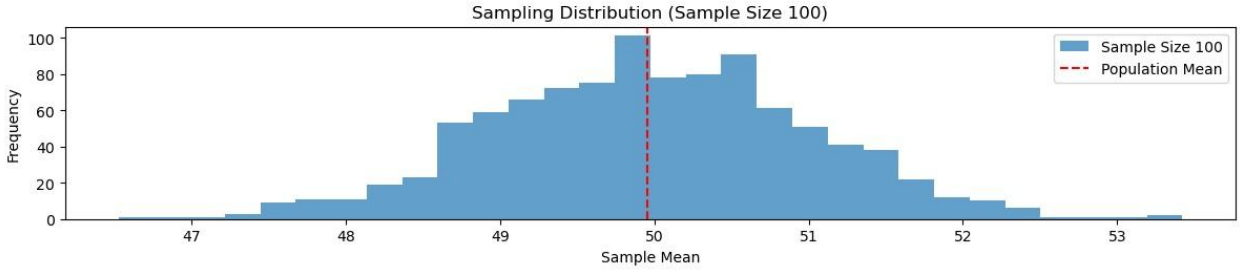
for size in sample_sizes:     sample_means[size] = []
for _ in range(num_samples):          sample =
np.random.choice(population, size=size, replace=False)
        sample_means[size].append(np.mean(sample))

# Step 3: Plotting sampling distributions
plt.figure(figsize=(12, 8))

for i, size in enumerate(sample_sizes):
plt.subplot(len(sample_sizes), 1, i+1)
    plt.hist(sample_means[size],  bins=30,  alpha=0.7,  label=f'Sample
Size {size}')
    plt.axvline(np.mean(population), color='red', linestyle='dashed',
linewidth=1.5, label='Population Mean')
    plt.title(f'Sampling Distribution (Sample Size {size})')
plt.xlabel('Sample Mean')     plt.ylabel('Frequency')
plt.legend() plt.tight_layout()
plt.show()
```


Sampling Distribution (Sample Size 30)
Sample Size 30
Population Mean
Frequency
Sample Mean


Sampling Distribution (Sample Size 50)
Sample Size 50
Population Mean
Frequency
Sample Mean


Sampling Distribution (Sample Size 100)
Sample Size 100
Population Mean
Frequency
Sample Mean

In [ ]:
```
Exp No:2 Upload and Analyze the data set given in csv format and perform data
preprocessing and visualization.
```

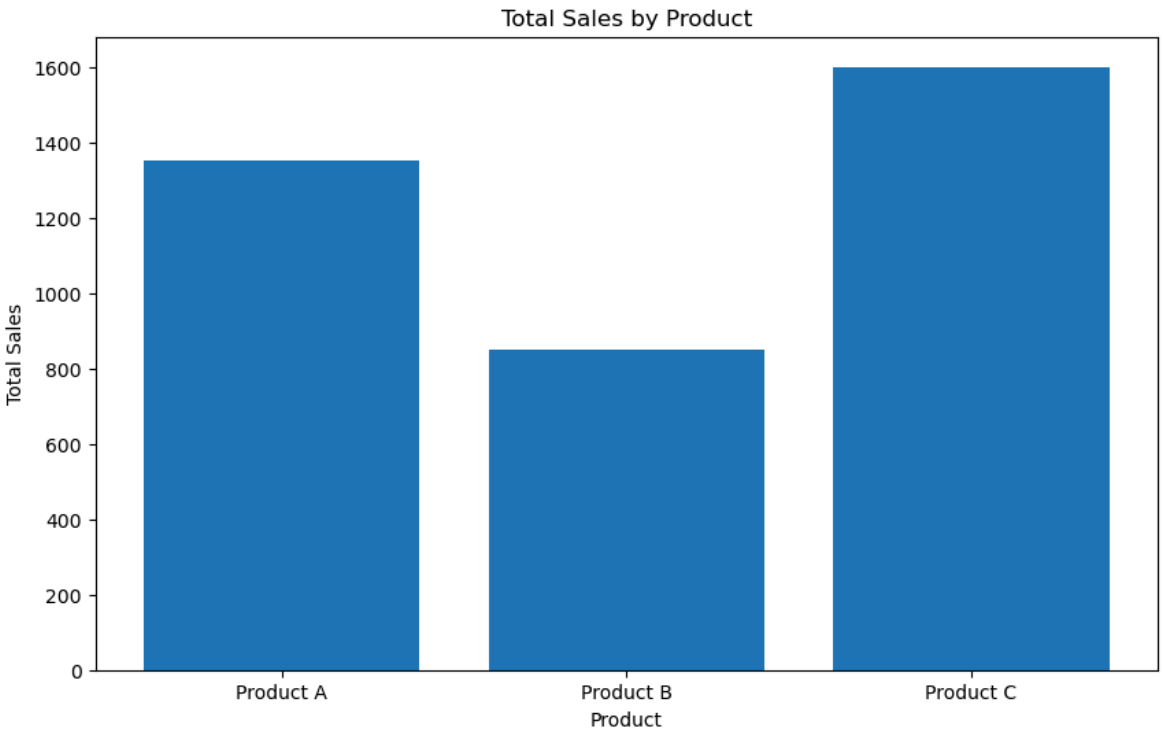
In [3]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
file_path = r"C:\Users\jsudi\OneDrive\Documents\Book1.csv"
df = pd.read_csv(file_path)
print(df.head())
print("\nMissing Values:")
print(df.isnull().sum())
df['Sales'] = df['Sales'].fillna(df['Sales'].mean())
df.dropna(subset=['Product', 'Quantity', 'Region'], inplace=True)
print("\nSummary Statistics:")
print(df.describe())
product_summary = df.groupby('Product').agg({
    'Sales': 'sum',
    'Quantity': 'sum'
}).reset_index()
print("\nProduct Summary:")
print(product_summary)
plt.figure(figsize=(10, 6))
plt.bar(product_summary['Product'], product_summary['Sales'])
plt.xlabel('Product')
plt.ylabel('Total Sales')
plt.title('Total Sales by Product')
plt.show()
df['Date'] = pd.to_datetime(df['Date'], dayfirst=True)
sales_over_time = df.groupby('Date')['Sales'].sum().reset_index()
plt.figure(figsize=(10, 6))
plt.plot(sales_over_time['Date'], sales_over_time['Sales'])
plt.xlabel('Date')
plt.ylabel('Total Sales')
plt.title('Sales Over Time')
plt.xticks(rotation=45)
plt.show()
pivot_table = df.pivot_table(
    values='Sales',
    index='Region',
    columns='Product',
    aggfunc='sum',
    fill_value=0
)
print("\nPivot Table:")
print(pivot_table)
correlation_matrix = df.corr(numeric_only=True)
print("\nCorrelation Matrix:")
print(correlation_matrix)
plt.figure(figsize=(8, 6))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm')
plt.title('Correlation Matrix')
plt.show()
```

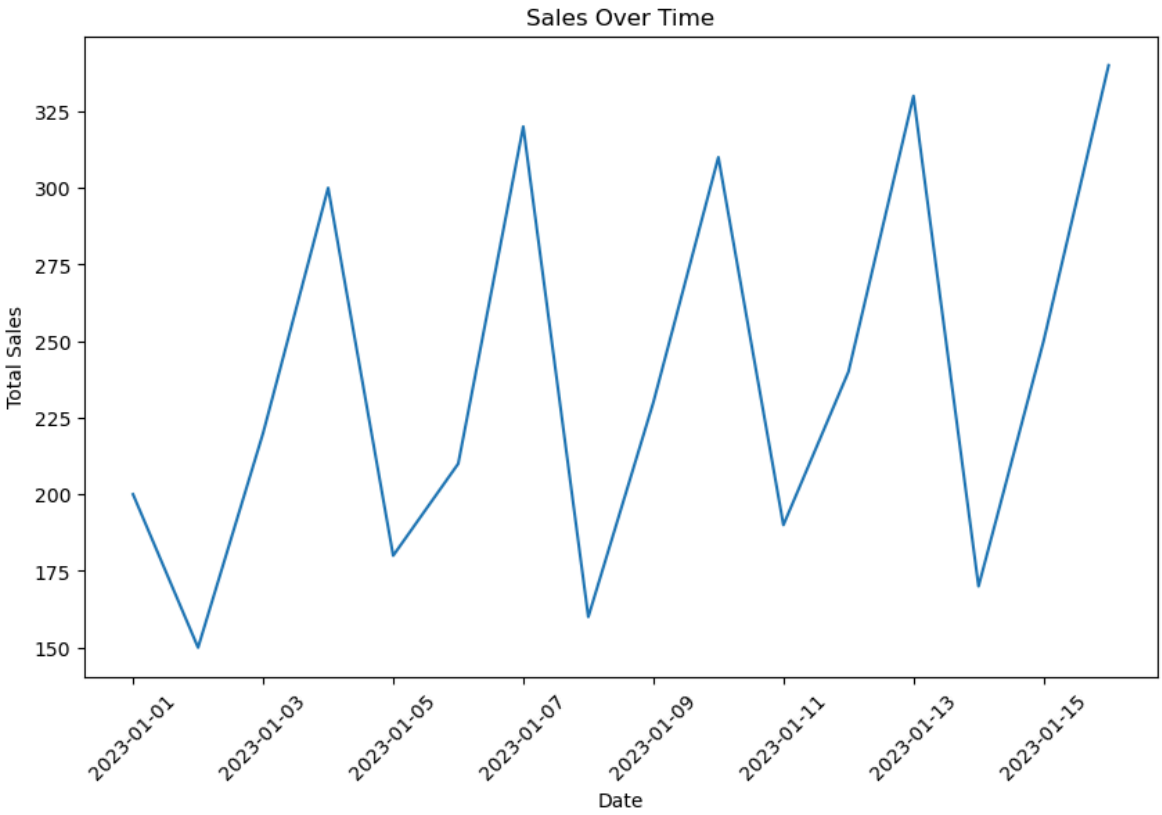
```
         Date    Product  Sales  Quantity Region
0  01-01-2023  Product A    200         4  North
1  02-01-2023  Product B    150         3  South
2  03-01-2023  Product A    220         5  North
3  04-01-2023  Product C    300         6   East
4  05-01-2023  Product B    180         4   West

Missing Values:
Date        0
Product     0
Sales       0
Quantity    0
Region      0
dtype: int64

Summary Statistics:
            Sales   Quantity
count   16.000000  16.000000
mean   237.500000   5.375000
std     64.031242   1.746425
min    150.000000   3.000000
25%    187.500000   4.000000
50%    225.000000   5.500000
75%    302.500000   7.000000
max    340.000000   8.000000

Product Summary:
     Product  Sales  Quantity
0  Product A   1350        33
1  Product B    850        17
2  Product C   1600        36
```



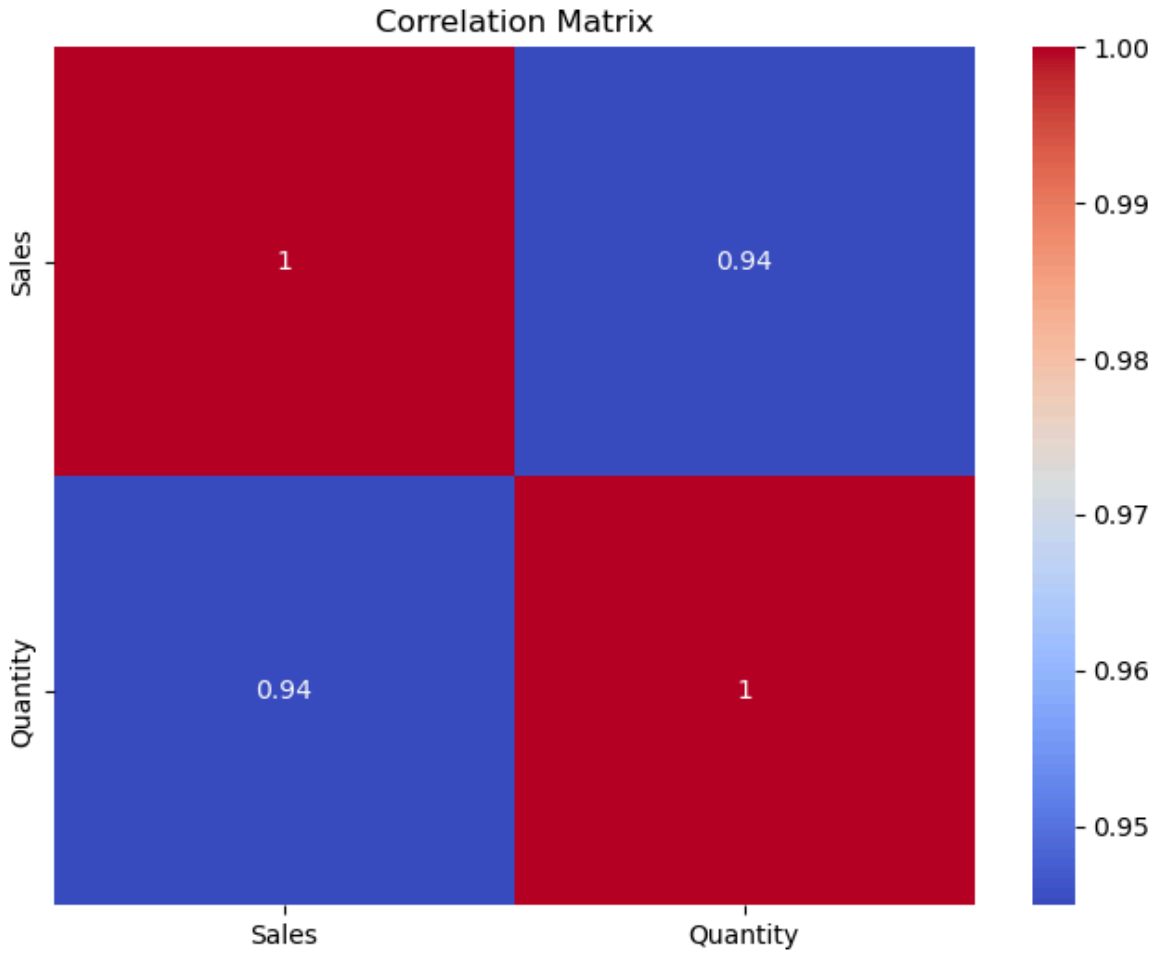


Pivot Table:

Product	Product A	Product B	Product C
Region			
East	0	0	1600
North	1350	0	0
South	0	480	0
West	0	370	0

Correlation Matrix:

	Sales	Quantity
Sales	1.000000	0.944922
Quantity	0.944922	1.000000


Correlation Matrix
Sales
Quantity
1
0.94
0.94
1
Sales
Quantity
1.00
0.99
0.98
0.97
0.96
0.95

In [ ]:

Name: S SHUVANTH Register Number: 250701708

Experiment Name:Handling Missing and Inappropriate Data in a Dataset

```
import numpy as np
import pandas as pd
df = pd.read_csv("Hotel_Dataset.csv")
print("--- Initial Duplicate Identification ---")
print(df.duplicated())
print("\n--- Dataframe Info Summary ---")
df.info()
df.drop_duplicates(inplace=True) index =
np.array(list(range(0, len(df))))
df.set_index(index, inplace=True)
df.drop(["Age_Group.1"], axis=1, inplace=True)
df.CustomerID.loc[df.CustomerID < 0] = np.nan
df.Bill.loc[df.Bill < 0] = np.nan
df.EstimatedSalary.loc[df.EstimatedSalary < 0] = np.nan
df["NoOfPax"].loc[(df["NoOfPax"] < 1) | (df["NoOfPax"] > 20)] = np.nan
print("\n--- Unique Categorical Value Sets ---") print("Age Groups:",
df.Age_Group.unique()) print("Hotels:", df.Hotel.unique())
df.Hotel.replace(["Ibys"], "Ibis", inplace=True)
df.FoodPreference.replace(["Vegetarian", "veg"], "Veg", inplace=True)
df.FoodPreference.replace(["non-Veg"], "Non-Veg", inplace=True)
df.EstimatedSalary.fillna(round(df.EstimatedSalary.mean()),
inplace=True)
df.NoOfPax.fillna(round(df.NoOfPax.median()), inplace=True)
df["Rating(1-5)"].fillna(round(df["Rating(1-5)"].median()),
inplace=True)
df.Bill.fillna(round(df.Bill.mean()), inplace=True)
print("\n--- Final Cleaned Dataset (df) ---")
print(df)
--- Initial Duplicate Identification ---
0     False
1     False
2     False
3     False
4     False
5     False
6     False
7     False
8     False
9     True
10     False
dtype: bool
--- Dataframe Info Summary ---

<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 11 entries, 0 to 10
Data columns (total 9 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
0    CustomerID       11 non-null     int64
1    Age_Group        11 non-null     object
2    Rating(1-5)      11 non-null     int64
3    Hotel            11 non-null     object
4    FoodPreference   11 non-null     object
5    Bill             11 non-null     int64
6    NoOfPax          11 non-null     int64
7    EstimatedSalary  11 non-null     int64  8   Age_Group.1      11
non-null     object
dtypes: int64(5), object(4)
memory usage: 924.0+ bytes

--- Unique Categorical Value Sets ---
Age Groups: ['20-25' '30-35' '25-30' '35+']
Hotels: ['Ibis' 'LemonTree' 'RedFox' 'Ibys']

 --- Final Cleaned Dataset (df) ---
   CustomerID Age_Group  Rating(1-5)      Hotel FoodPreference    Bill


          1.0     20-25            4       Ibis            Veg  1300.0

\
0

1          2.0     30-35            5  LemonTree        Non-Veg  2000.0

2          3.0     25-30            6     RedFox            Veg  1322.0

3          4.0     20-25           -1  LemonTree            Veg  1234.0

4          5.0       35+            3       Ibis            Veg   989.0

5          6.0       35+            3       Ibis        Non-Veg  1909.0

6          7.0       35+            4     RedFox            Veg  1000.0

7          8.0     20-25            7  LemonTree            Veg  2999.0

8          9.0     25-30            2       Ibis        Non-Veg  3456.0

9          10.0     30-35            5     RedFox        Non-Veg  1801.0
```

	NoOfPax	EstimatedSalary
0	2.0	40000.0
1	3.0	59000.0
2	2.0	30000.0
3	2.0	120000.0
4	2.0	45000.0

```
5      2.0         122220.0
6      2.0          21122.0
7      2.0         345673.0
8      3.0          96755.0
9      4.0          87777.0

C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:12:
FutureWarning: ChainedAssignmentError: behaviour will change in pandas
3.0! You are setting values through chained assignment. Currently this
works in certain cases, but when using Copy-on-Write (which will
become the default behaviour in pandas 3.0) this will never work to
update the original DataFrame or Series, because the intermediate
object on which we are setting values will behave as a copy. A typical
example is when you are setting values in a column of a
DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the
assignment in a single step and ensure this keeps updating the
original `df`.

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy

  df.CustomerID.loc[df.CustomerID < 0] = np.nan
C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:12:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy
  df.CustomerID.loc[df.CustomerID < 0] = np.nan
C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:13:
FutureWarning: ChainedAssignmentError: behaviour will change in pandas
3.0! You are setting values through chained assignment. Currently this
works in certain cases, but when using Copy-on-Write (which will
become the default behaviour in pandas 3.0) this will never work to
update the original DataFrame or Series, because the intermediate
object on which we are setting values will behave as a copy. A typical
example is when you are setting values in a column of a
DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the
```

```
assignment in a single step and ensure this keeps updating the
original `df`.

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy

  df.Bill.loc[df.Bill < 0] = np.nan
C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:13:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy
df.Bill.loc[df.Bill < 0] = np.nan
C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:14:
FutureWarning: ChainedAssignmentError: behaviour will change in pandas
3.0! You are setting values through chained assignment. Currently this
works in certain cases, but when using Copy-on-Write (which will
become the default behaviour in pandas 3.0) this will never work to
update the original DataFrame or Series, because the intermediate
object on which we are setting values will behave as a copy. A typical
example is when you are setting values in a column of a
DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the
assignment in a single step and ensure this keeps updating the
original `df`.

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy

  df.EstimatedSalary.loc[df.EstimatedSalary < 0] = np.nan
C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:14:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy
  df.EstimatedSalary.loc[df.EstimatedSalary < 0] = np.nan
C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:15:
FutureWarning: ChainedAssignmentError: behaviour will change in pandas
3.0! You are setting values through chained assignment. Currently this
```

```
works in certain cases, but when using Copy-on-Write (which will
become the default behaviour in pandas 3.0) this will never work to
update the original DataFrame or Series, because the intermediate
object on which we are setting values will behave as a copy. A
typical example is when you are setting values in a column of a
DataFrame, like:

df["col"][row_indexer] = value

Use `df.loc[row_indexer, "col"] = values` instead, to perform the
assignment in a single step and ensure this keeps updating the
original `df`.

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy

  df["NoOfPax"].loc[(df["NoOfPax"] < 1) | (df["NoOfPax"] > 20)] =
np.nan
C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:15:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy
  df["NoOfPax"].loc[(df["NoOfPax"] < 1) | (df["NoOfPax"] > 20)] =
np.nan
C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:19:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df.Hotel.replace(["Ibys"], "Ibis", inplace=True)
C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:20:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.
```

```
For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df.FoodPreference.replace(["Vegetarian", "veg"], "Veg",
inplace=True)
C:\Users\swath\AppData\Local\Temp\ipykernel_28208\2624405208.py:24:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df["Rating(1-5)"].fillna(round(df["Rating(1-5)"].median()),
inplace=True)
```

Name: S SHUVANTH Register Number: 250701708
Experiment Name:Experiment to Understand Data Preprocessing in Data Science

```
import numpy as np
import pandas as pd

df = pd.read_csv('pre-process_datasample.csv')

df['Country'].fillna(df['Country'].mode(), inplace=True)
df['Age'].fillna(df['Age'].median(), inplace=True)
df['Salary'].fillna(round(df['Salary'].mean()), inplace=True)

dummy_countries = pd.get_dummies(df['Country'])
updated_dataset = pd.concat([dummy_countries, df.iloc[:, 1:4]],
axis=1)
updated_dataset['Purchased'].replace(['No', 'Yes'], inplace=True)

print(updated_dataset)
```

```
   France  Germany  Spain   Age   Salary Purchased
0    True    False  False  44.0  72000.0        No
1    False    False   True  27.0  48000.0         No
2    False     True  False  30.0  54000.0         No
3    False    False   True  38.0  61000.0         No
4    False     True  False  40.0  63778.0         No
5    True    False  False  35.0  58000.0        No
6    False    False   True  38.0  52000.0         No
7    True    False  False  48.0  79000.0        No
8    False     True  False  50.0  83000.0         No
9    True    False  False  37.0  67000.0        No
```

```
C:\Users\swath\AppData\Local\Temp\ipykernel_22860\328809920.py:6:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df['Country'].fillna(df['Country'].mode(), inplace=True)
C:\Users\swath\AppData\Local\Temp\ipykernel_22860\328809920.py:7:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
```

```
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df['Age'].fillna(df['Age'].median(), inplace=True)
C:\Users\swath\AppData\Local\Temp\ipykernel_22860\328809920.py:8:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  df['Salary'].fillna(round(df['Salary'].mean()), inplace=True)
C:\Users\swath\AppData\Local\Temp\ipykernel_22860\328809920.py:12:
FutureWarning: Series.replace without 'value' and with non-dict-like
'to_replace' is deprecated and will raise in a future version.
Explicitly specify the new values instead.
updated_dataset['Purchased'].replace(['No', 'Yes'], inplace=True)
C:\Users\swath\AppData\Local\Temp\ipykernel_22860\328809920.py:12:
FutureWarning: A value is trying to be set on a copy of a DataFrame or
Series through chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method will never
work because the intermediate object on which we are setting values
always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)', try
using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace on the
original object.


  updated_dataset['Purchased'].replace(['No', 'Yes'], inplace=True)
```

Name: S SHUVANTH Register Number: 250701708

Experiment Name:Outliers Detection

```
import numpy as np import
seaborn as sns import
matplotlib.pyplot as plt

array = np.array([27, 50, 44, 6, 58, 61, 23, 86, 67, 20, 75, 7, 79,
61, 90, 54])

def outDetection(arr):
    Q1, Q3 = np.percentile(arr, [25, 75])
IQR = Q3 - Q1     lr = Q1 - (1.5 * IQR)
ur = Q3 + (1.5 * IQR)     return lr, ur

lr, ur = outDetection(array)

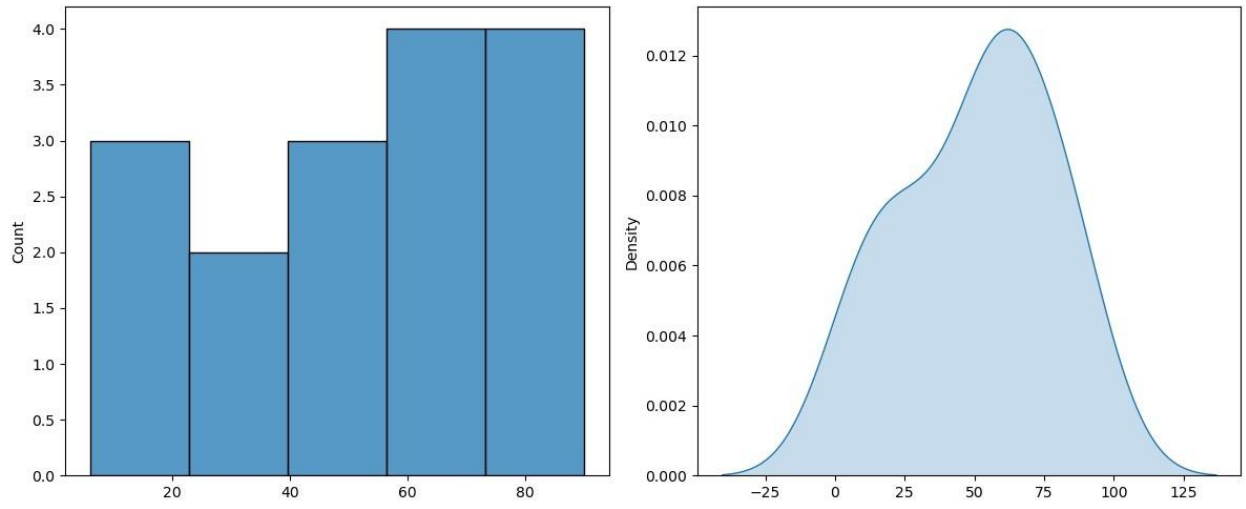
final_array = array[(array >= lr) & (array <= ur)]

plt.figure(figsize=(12, 5))

plt.subplot(1, 2, 1)
sns.histplot(final_array, kde=False)

plt.subplot(1, 2, 2)
sns.kdeplot(final_array, fill=True)

plt.tight_layout()
plt.show()
```



Name: S SHUVANTH Register Number: 250701708 aling

```
import numpy as np
import pandas as pd
from sklearn.impute import SimpleImputer from
sklearn.preprocessing import OneHotEncoder, StandardScaler,
MinMaxScaler df = pd.read_csv('pre-process_datasample.csv')
df['Country'] = df['Country'].fillna(df['Country'].mode()[0])

features = df.iloc[:, :-1].values
label = df.iloc[:, -1].values

age = SimpleImputer(strategy="mean", missing_values=np.nan)
salary = SimpleImputer(strategy="mean", missing_values=np.nan)

features[:, 1:2] = age.fit_transform(features[:, 1:2])
features[:, 2:3] = salary.fit_transform(features[:, 2:3])

oh = OneHotEncoder(sparse_output=False) Country =
oh.fit_transform(features[:, 0:1]) final_set =
np.concatenate((Country, features[:, 1:3]), axis=1)

sc = StandardScaler() feat_standard_scaler =
sc.fit_transform(final_set)
print(feat_standard_scaler)

mms = MinMaxScaler(feature_range=(0, 1))
feat_minmax_scaler = mms.fit_transform(final_set)
print(feat_minmax_scaler)
[[ 1.22474487e+00 -6.54653671e-01 -6.54653671e-01  7.58874362e-01
   7.49473254e-01]

 [-8.16496581e-01 -6.54653671e-01  1.52752523e+00 -1.71150388e+00
  -1.43817841e+00]

 [-8.16496581e-01  1.52752523e+00 -6.54653671e-01 -1.27555478e+00
  -8.91265492e-01]

 [-8.16496581e-01 -6.54653671e-01  1.52752523e+00 -1.13023841e-01
  -2.53200424e-01]

 [-8.16496581e-01  1.52752523e+00 -6.54653671e-01  1.77608893e-01
   6.63219199e-16]

 [ 1.22474487e+00 -6.54653671e-01 -6.54653671e-01 -5.48972942e-01
  -5.26656882e-01]
```

```
[-8.16496581e-01 -6.54653671e-01  1.52752523e+00  0.00000000e+00
```

```
 -1.07356980e+00]
[ 1.22474487e+00 -6.54653671e-01 -6.54653671e-01  1.34013983e+00
  1.38753832e+00]
```

```
[-8.16496581e-01  1.52752523e+00 -6.54653671e-01  1.63077256e+00
```

```
   1.75214693e+00]
 [ 1.22474487e+00 -6.54653671e-01 -6.54653671e-01 -2.58340208e-01
   2.93712492e-01]]
[[1.         0.         0.         0.73913043 0.68571429]
 [0.         0.         1.         0.         0.        ]
 [0.         1.         0.         0.13043478 0.17142857]
 [0.         0.         1.         0.47826087 0.37142857]
[0.         1.         0.         0.56521739 0.45079365]
[1.         0.         0.         0.34782609 0.28571429]
[0.         0.         1.         0.51207729 0.11428571]
[1.         0.         0.         0.91304348 0.88571429]
[0.         1.         0.         1.         1.        ]
[1.         0.         0.         0.43478261 0.54285714]]
```

Name: S SHUVANTH Register Number: 250701708

Experiment Name:Experiment to understand EDA-Quantitative and Qualitative Analysis

```
import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
%matplotlib inline

tips = sns.load_dataset('tips')

print(tips.head())

sns.displot(tips['total_bill'], kde=True)
plt.show()

sns.displot(tips['total_bill'], kde=False)
plt.show()

sns.jointplot(x='tip', y='total_bill', data=tips)
plt.show()

sns.jointplot(x='tip', y='total_bill', data=tips, kind='reg')
plt.show()

sns.jointplot(x='tip', y='total_bill', data=tips, kind='hex')
plt.show()

sns.pairplot(tips)
plt.show()

print(tips['time'].value_counts())

sns.pairplot(tips, hue='time')
plt.show()

sns.pairplot(tips, hue='day')
plt.show()

sns.heatmap(tips.corr(numeric_only=True), annot=True)
plt.show()

sns.boxplot(y=tips['total_bill'])
plt.show()

sns.boxplot(y=tips['tip'])
plt.show()

sns.countplot(y=tips['day'])
plt.show()
```

```
sns.countplot(y=tips['sex'])
plt.show()

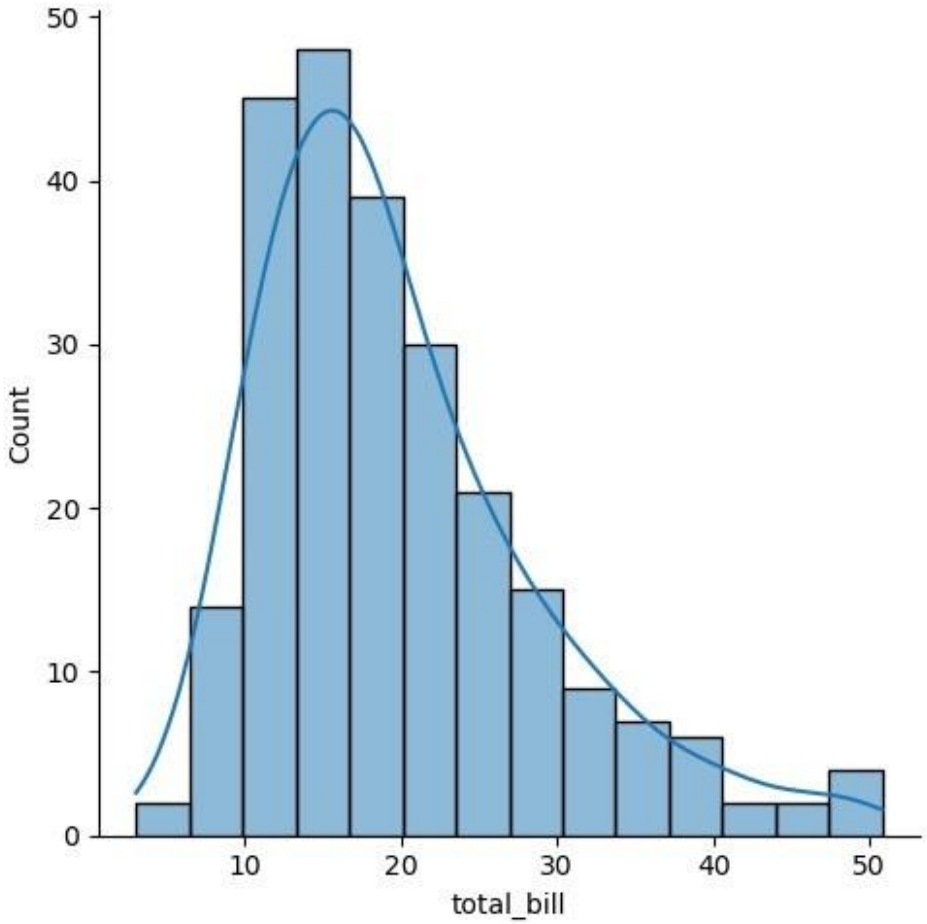
tips['sex'].value_counts().plot(kind='pie')
plt.show()

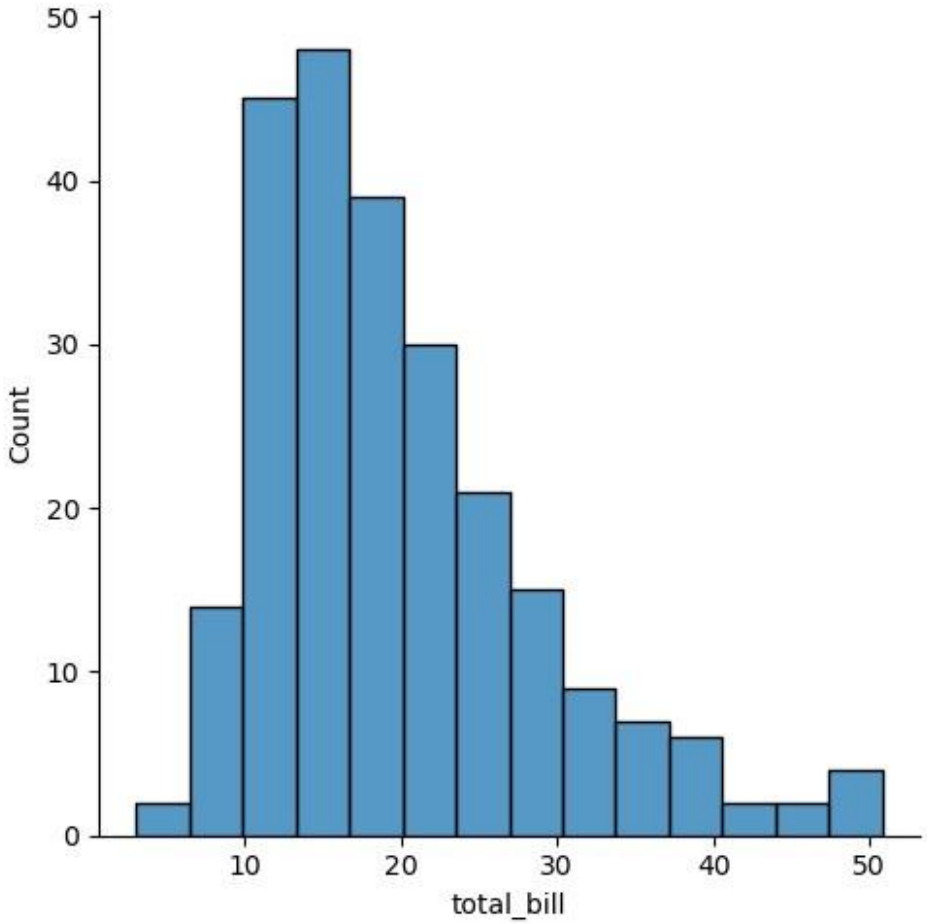
tips['sex'].value_counts().plot(kind='bar')
plt.show()

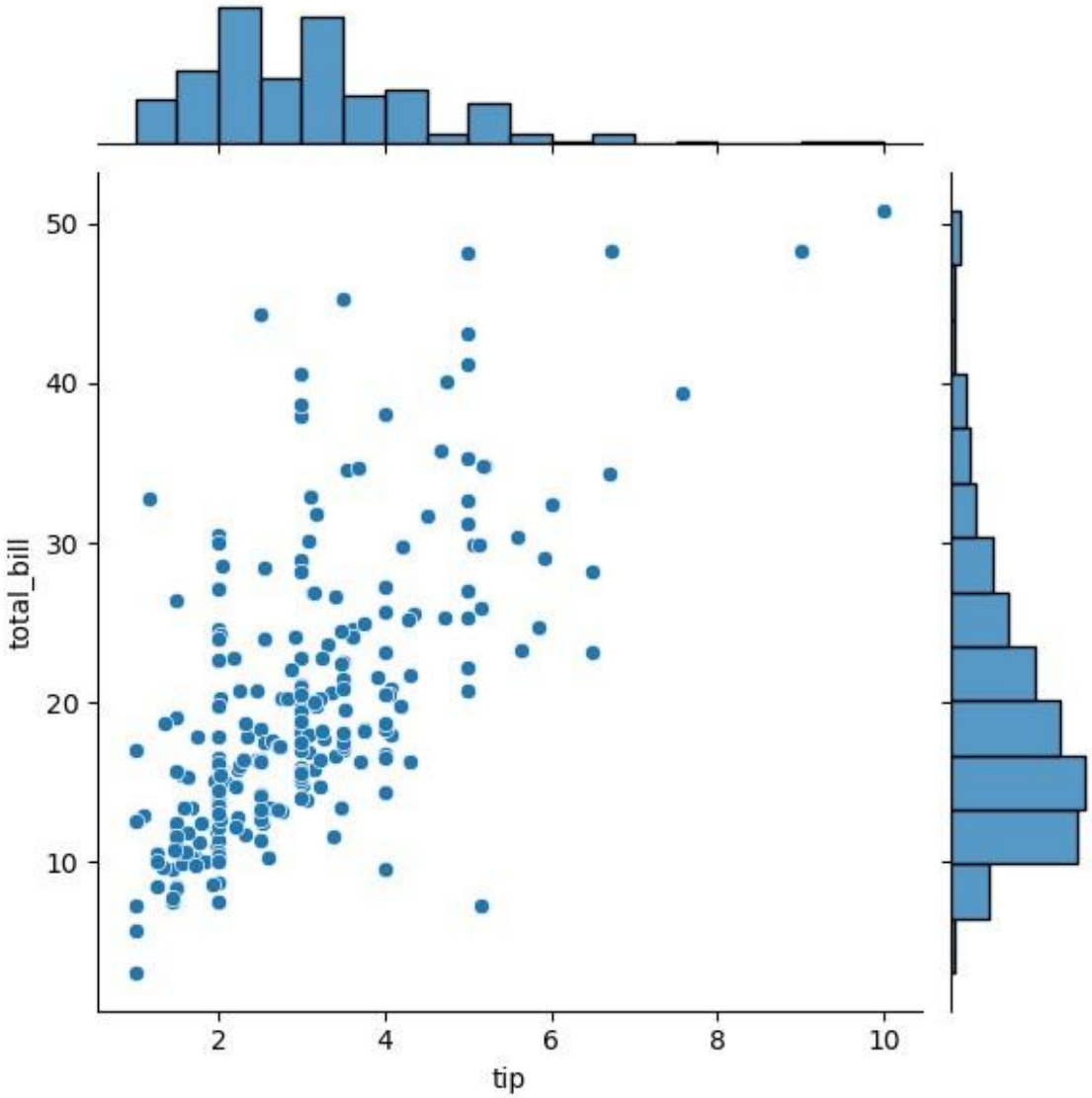
sns.countplot(x=tips['sex'])
plt.show()

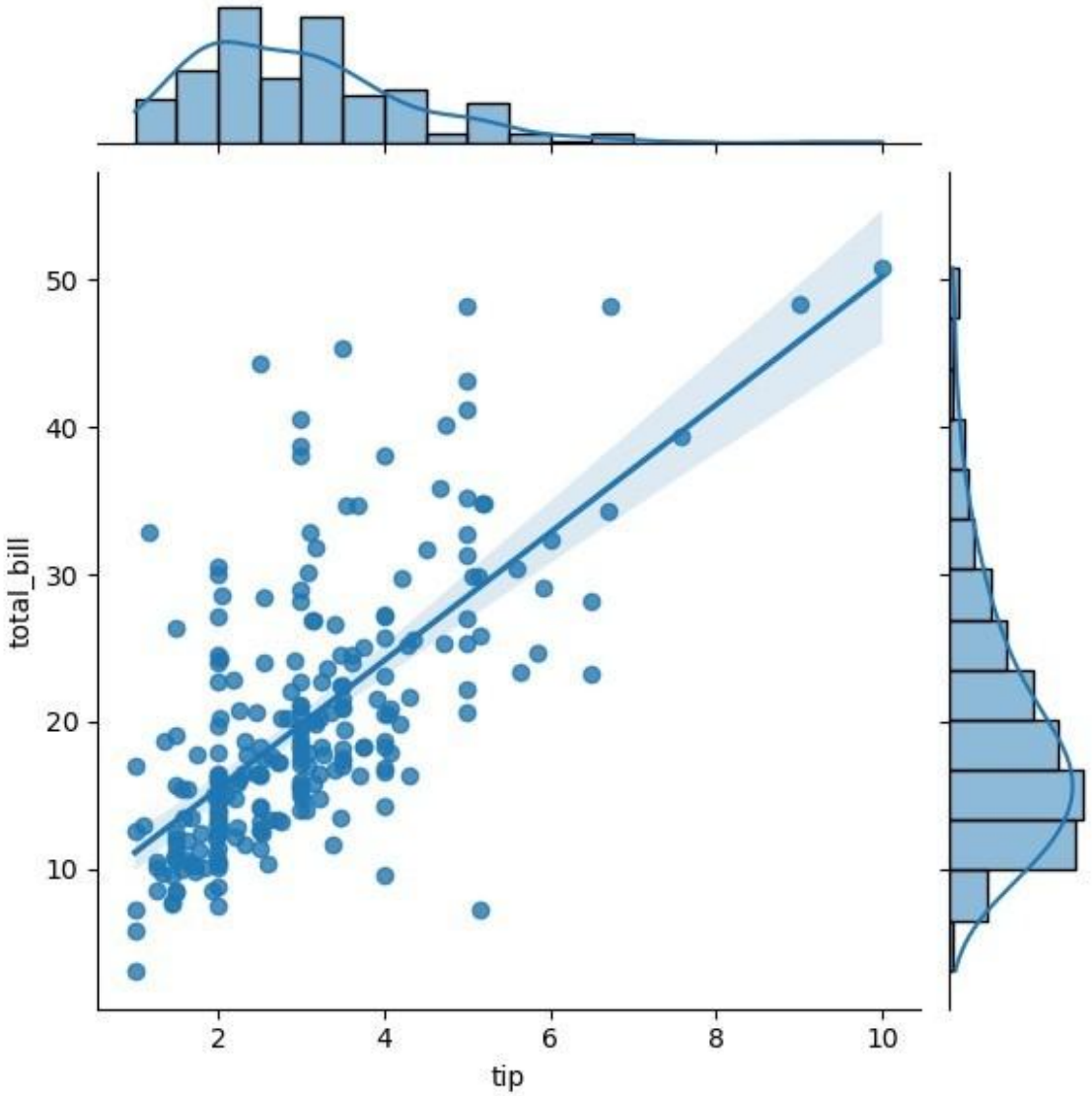
sns.countplot(data=tips[tips['time'] == 'Dinner'], x='day')
plt.show()
```

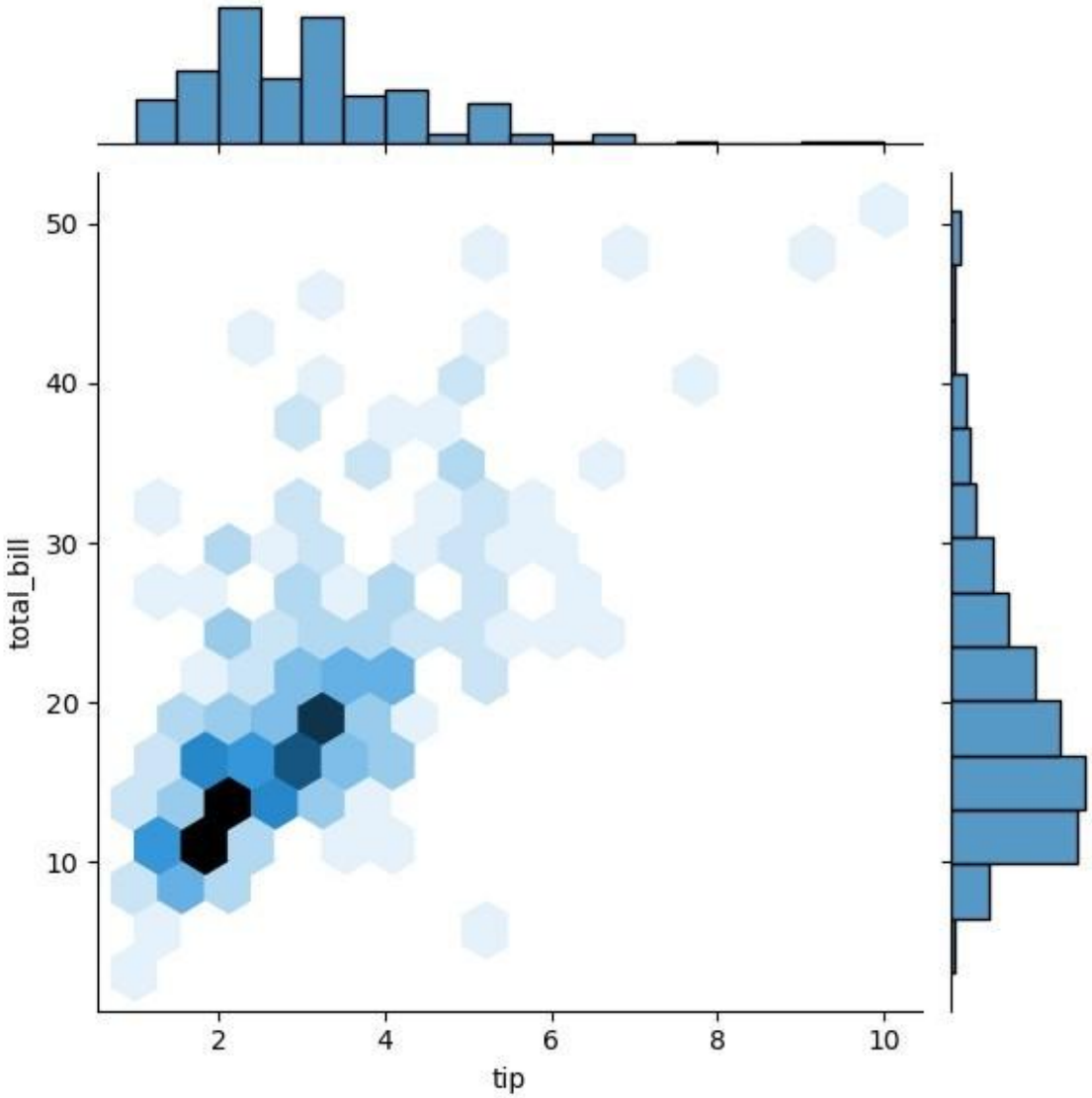
	total_bill	tip	sex	smoker	day	time	size
0	16.99	1.01	Female	No	Sun	Dinner	2
1	10.34	1.66	Male	No	Sun	Dinner	3
2	21.01	3.50	Male	No	Sun	Dinner	3
3	23.68	3.31	Male	No	Sun	Dinner	2
4	24.59	3.61	Female	No	Sun	Dinner	4

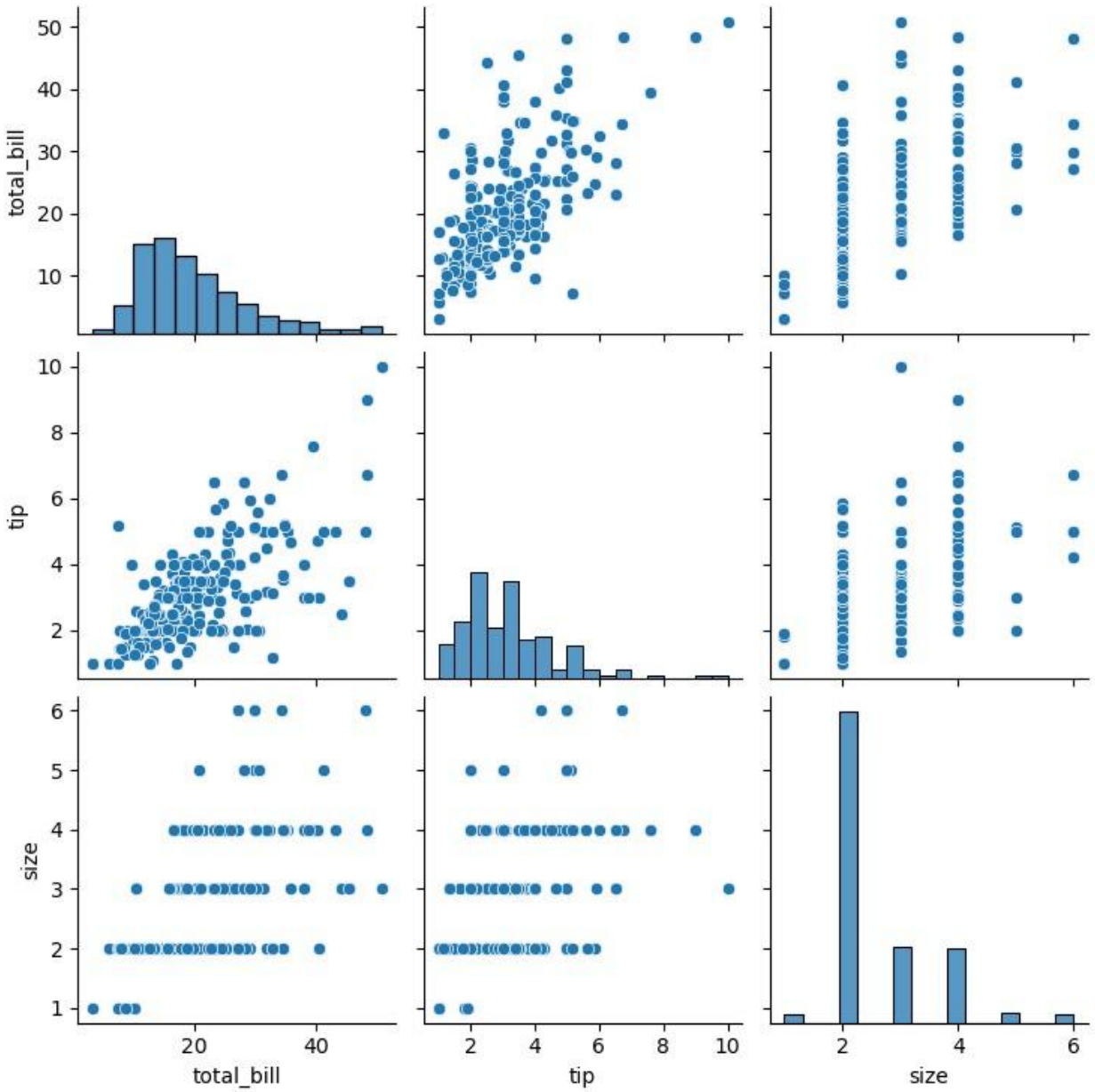

50
40
30
20
10
0
Count
10
20
30
40
50
total_bill


50
40
30
20
10
0
Count
10
20
30
40
50
total_bill


50
40
30
20
10
total_bill
2
4
6
8
10
tip

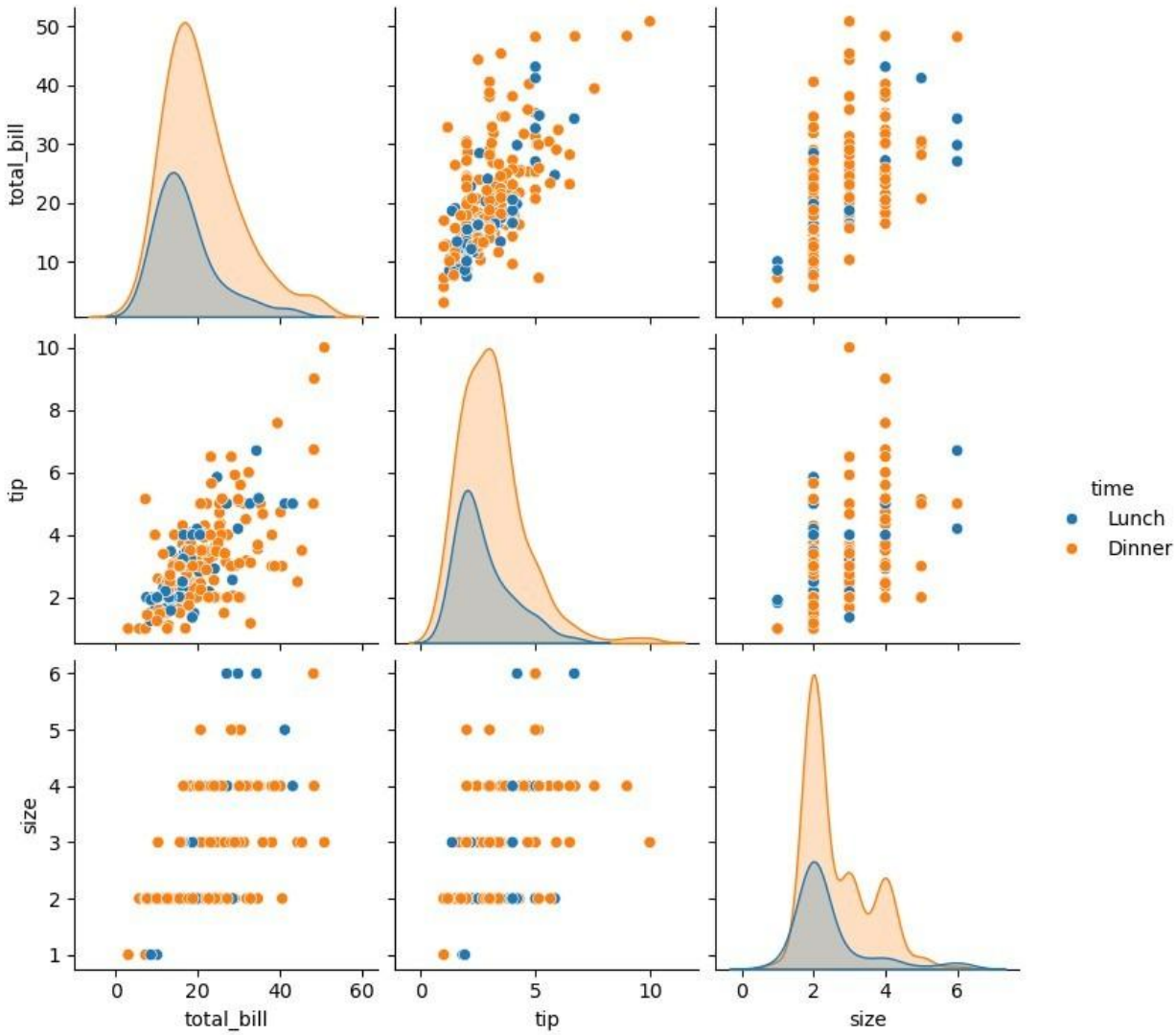

total_bill
tip
10
20
30
40
50
2
4
6
8
10

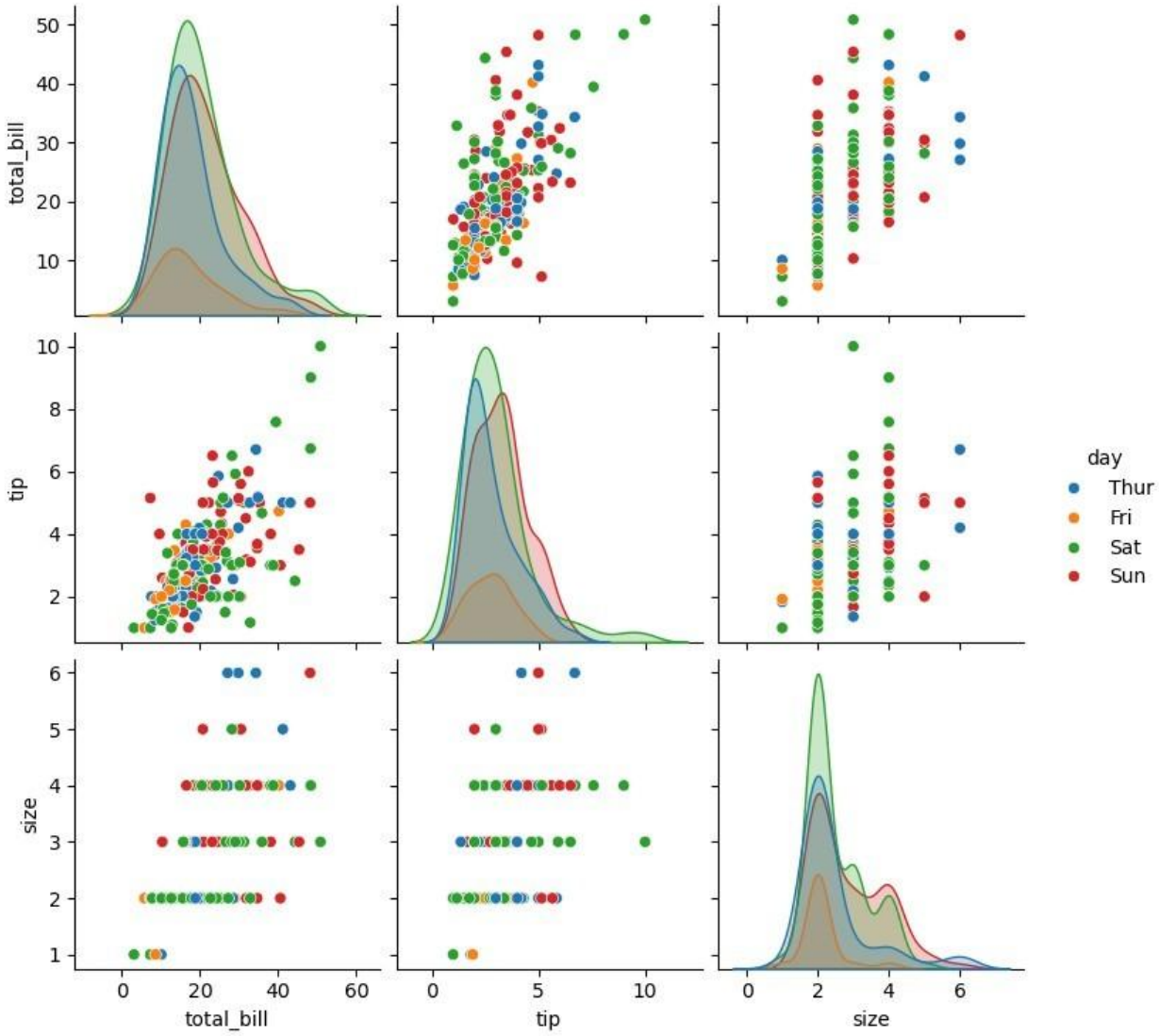

50
40
30
20
10
total_bill
2
4
6
8
10
tip

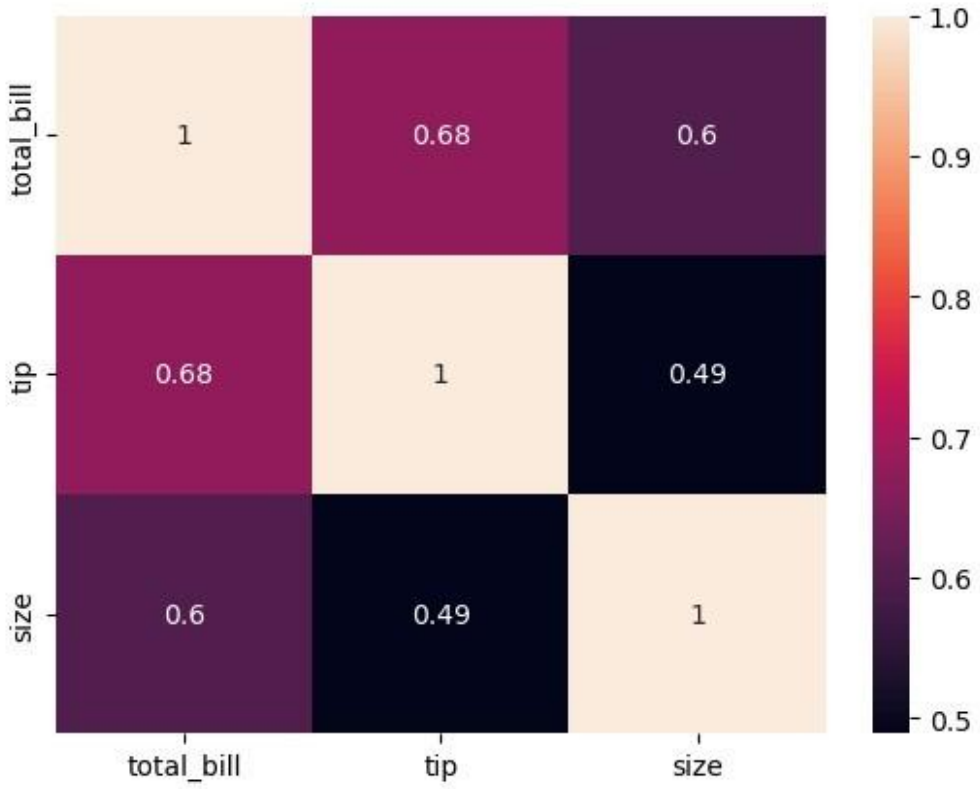


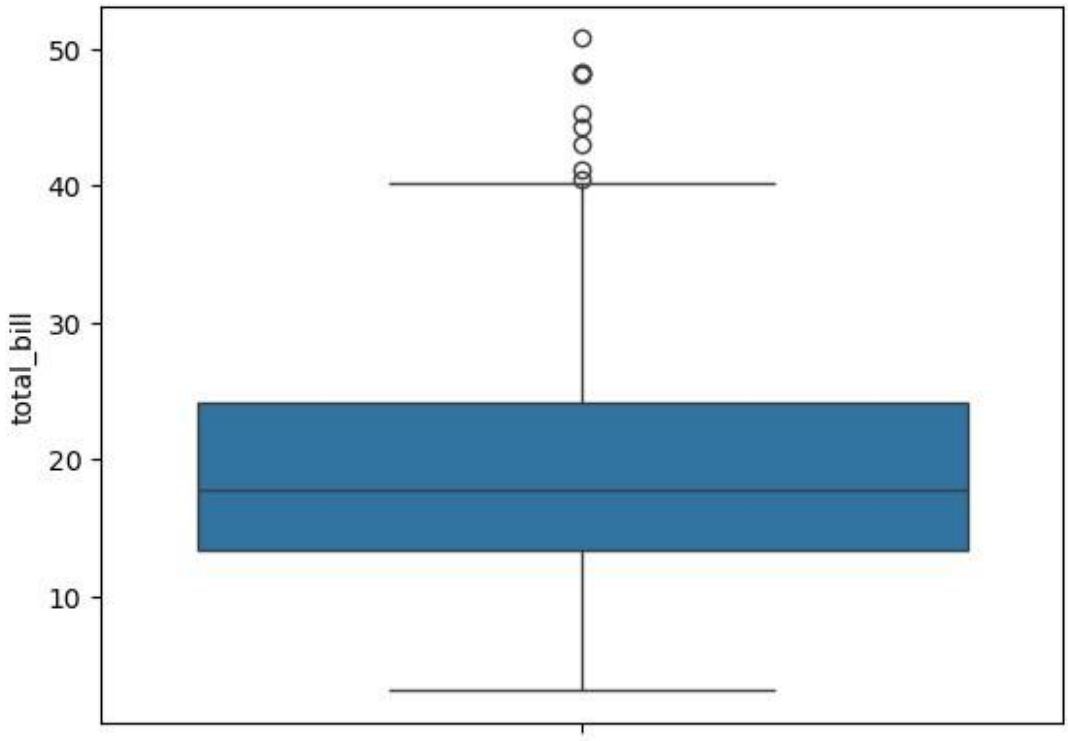

```
time

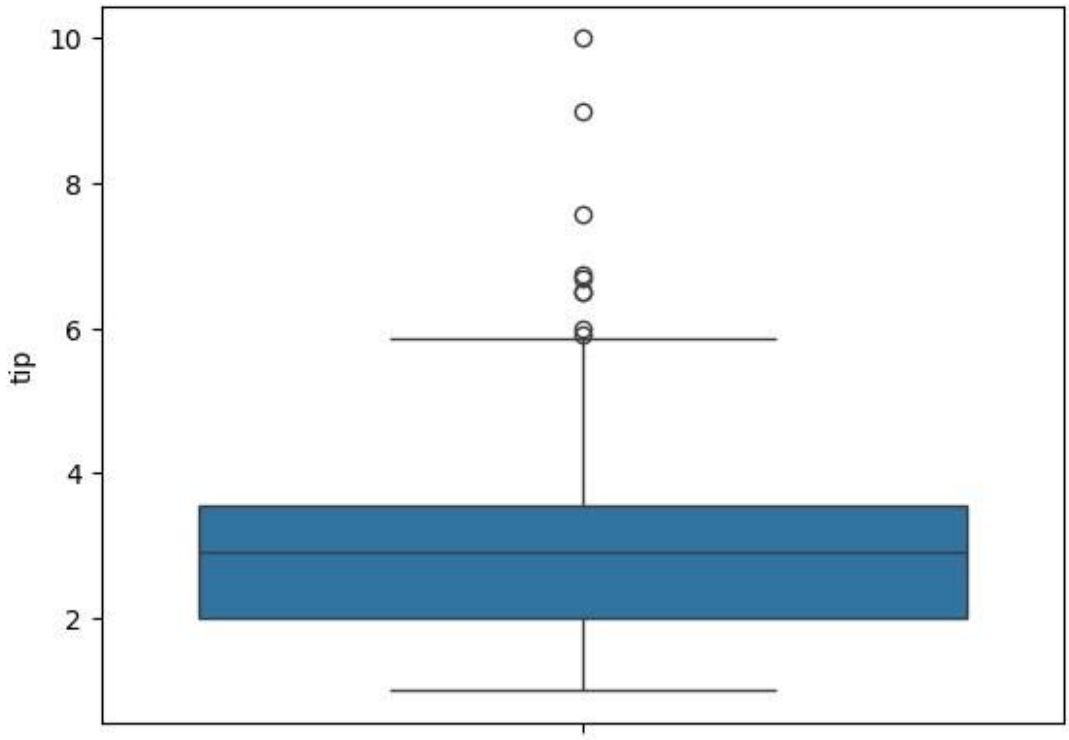
Dinner    176
Lunch      68
Name: count, dtype: int64
```

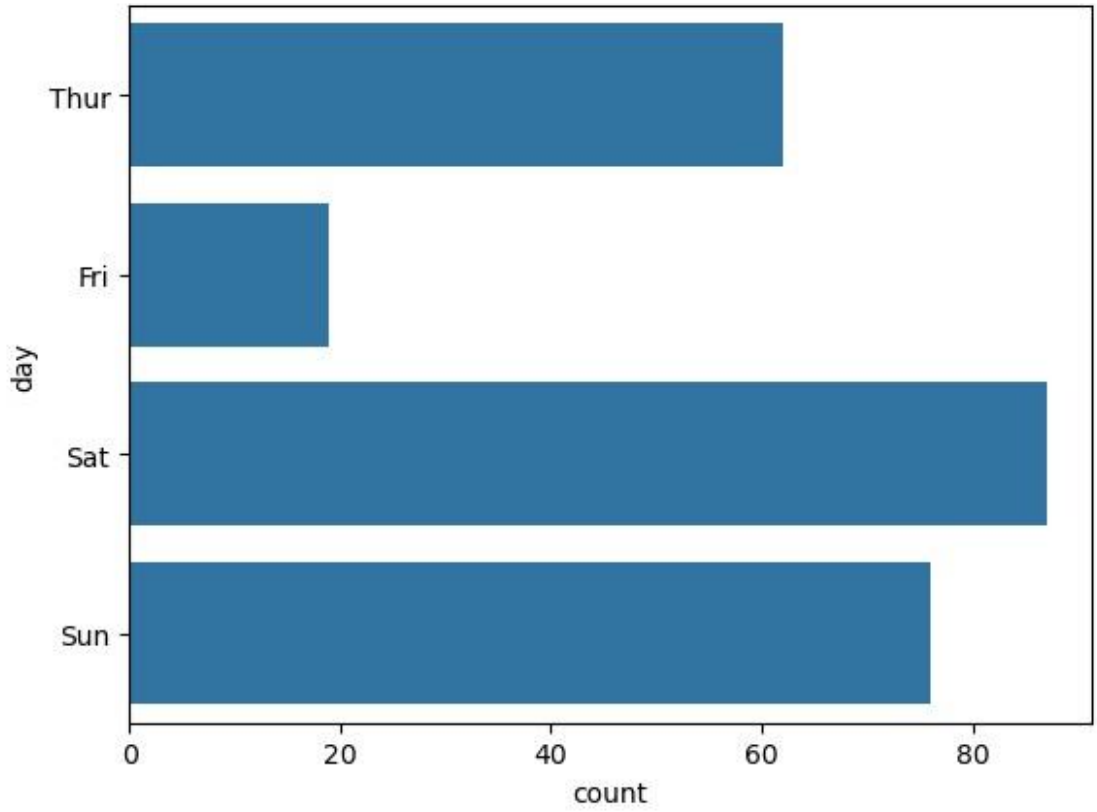

total_bill
tip
size
time
Lunch
Dinner

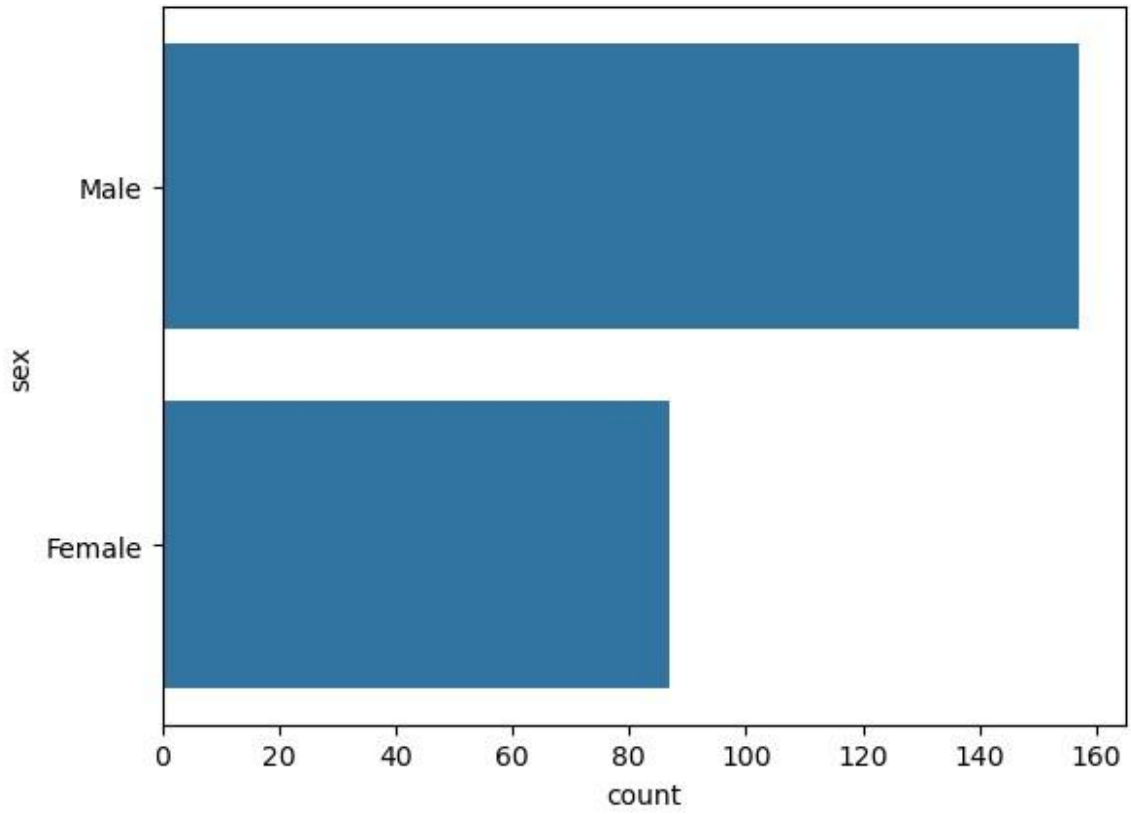

total_bill
tip
size
day
Thur
Fri
Sat
Sun

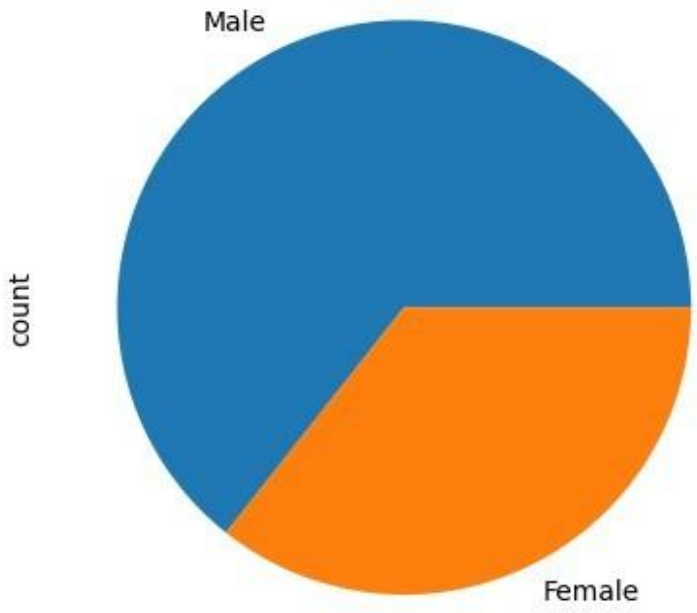

total_bill
tip
size
1
0.68
0.6
0.68
1
0.49
0.6
0.49
1
1.0
0.9
0.8
0.7
0.6
0.5
total_bill
tip
size

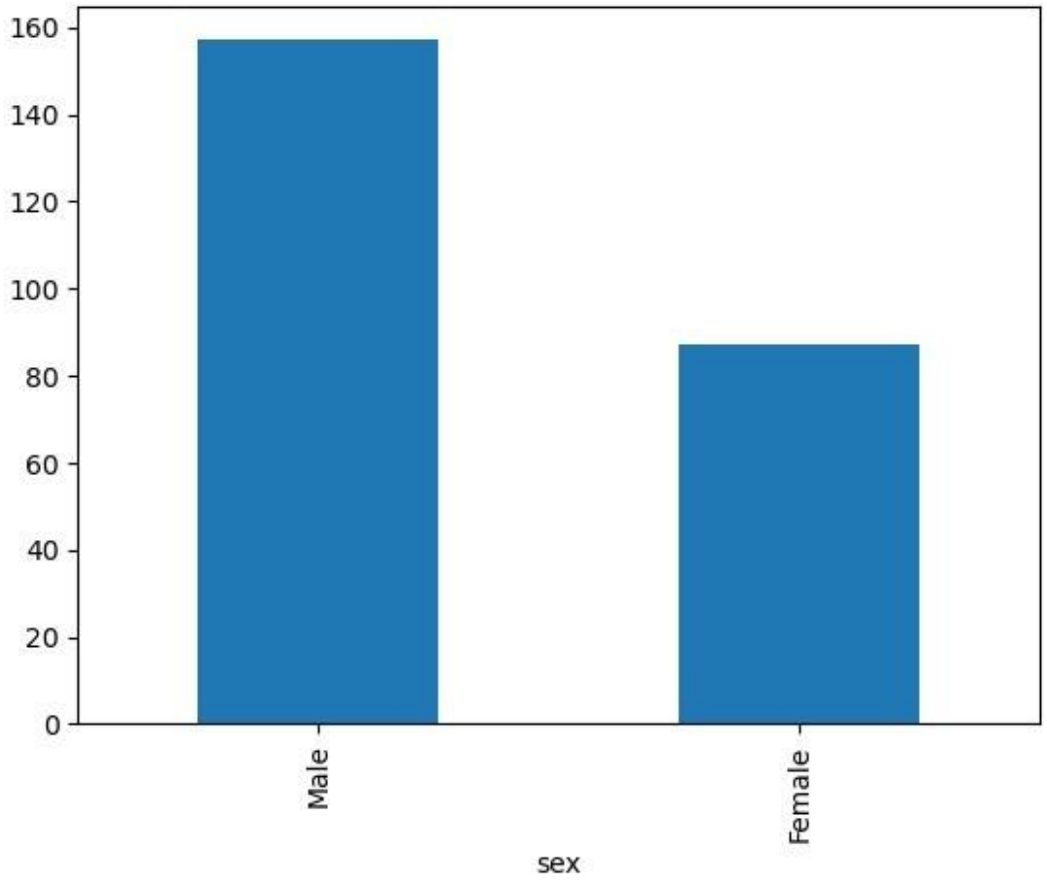

50
40
30
20
10
total_bill

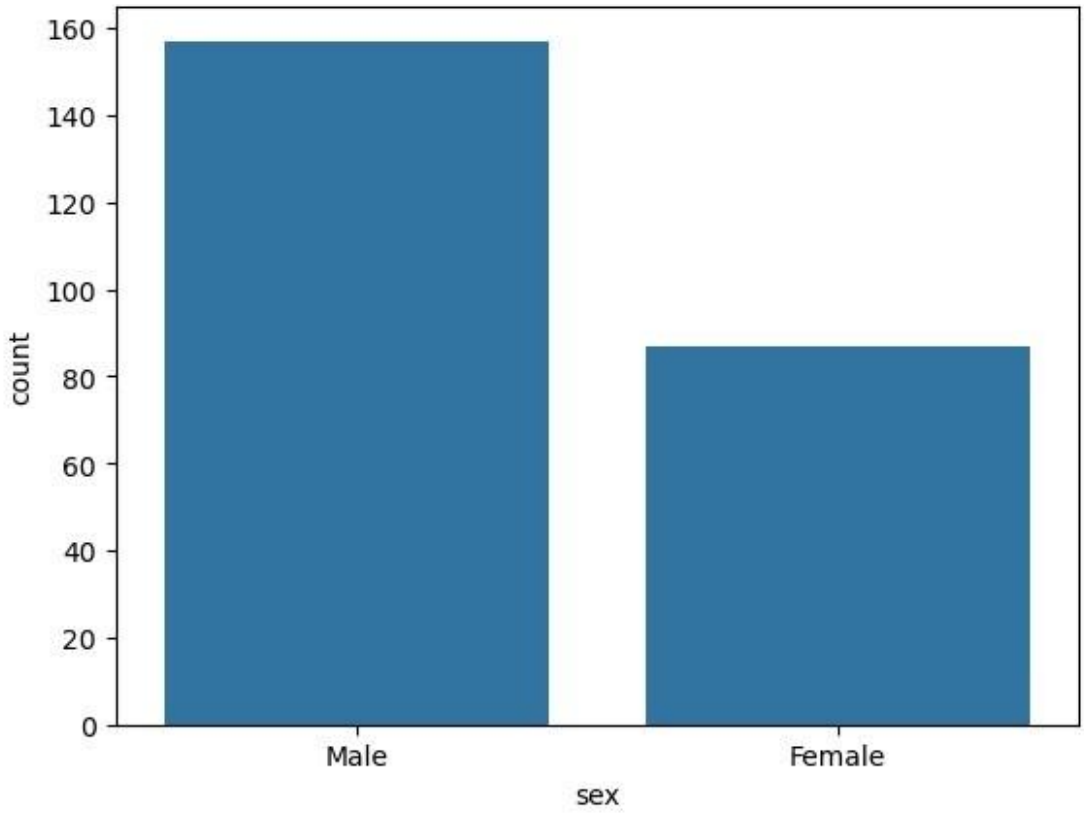

10
8
6
4
2
tip

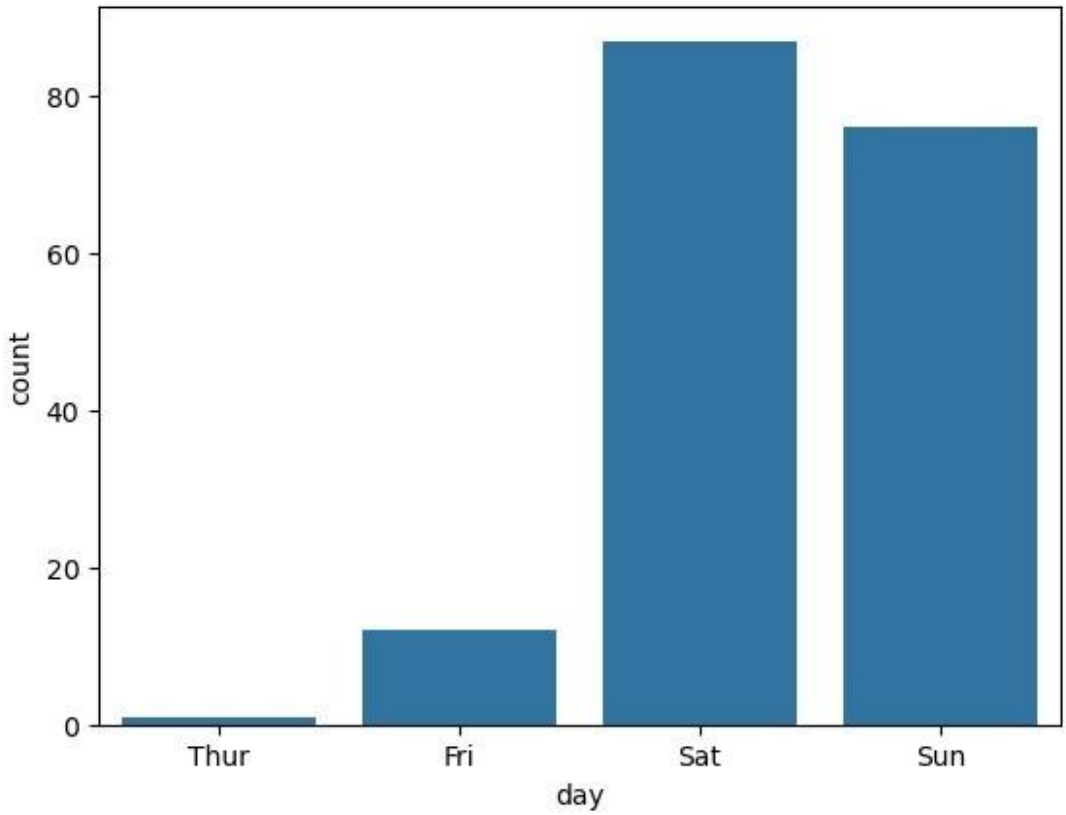

Thur
Fri
Sat
Sun
day
0
20
40
60
80
count


Male
Female
sex
0
20
40
60
80
100
120
140
160
count


Male
Female
count


160
140
120
100
80
60
40
20
0
Male
Female
sex


160
140
120
100
80
60
40
20
0
count
Male
Female
sex


80
60
40
20
0
count
Thur
Fri
Sat
Sun
day

Name: S SHUVANTH Register Number: 250701708 ar Regression

```
import numpy as np
import pandas as pd
df=pd.read_csv('Salary_data.csv')
df
```

```
    YearsExperience  Salary
0               1.1   39343
1               1.3   46205
2               1.5   37731
3               2.0   43525
4               2.2   39891
5               2.9   56642
6               3.0   60150
7               3.2   54445
8               3.2   64445
9               3.7   57189
10              3.9   63218
11              4.0   55794
12              4.0   56957
13              4.1   57081
14              4.5   61111
15              4.9   67938
16              5.1   66029
17              5.3   83088
18              5.9   81363
19              6.0   93940
20              6.8   91738
21              7.1   98273
22              7.9  101302
23              8.2  113812
24              8.7  109431
25              9.0  105582
26              9.5  116969
27              9.6  112635
28              10.3
                122391
29              10.5
                121872
```

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   YearsExperience  30 non-null     float64
1   Salary           30 non-null     int64
```

```
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

```
df.dropna(inplace=True)
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 2 columns):
 #   Column           Non-Null Count  Dtype
---  ------           --------------  -----
 0   YearsExperience  30 non-null     float64
1   Salary           30 non-null     int64
dtypes: float64(1), int64(1)
memory usage: 612.0 bytes
```

```
df.describe()
```

	YearsExperience	Salary
count	30.000000	30.000000
mean	5.313333	76003.000000
std	2.837888	27414.429785
min	1.100000	37731.000000
25%	3.200000	56720.750000
50%	4.700000	65237.000000
75%	7.700000	100544.750000
max	10.500000	122391.000000

```
features=df.iloc[:,[0]].values
label=df.iloc[:,[1]].values

from sklearn.model_selection import train_test_split
x_train,x_test,y_train,y_test=train_test_split(features,label,test_siz
e=0.2,random_state=42)

from sklearn.linear_model import LinearRegression
model=LinearRegression()
model.fit(x_train,y_train)
LinearRegression()
model.score(x_train,y_train)
0.9645401573418146
model.score(x_test,y_test)
0.9024461774180497
model.coef_
array([[9423.81532303]])
model.intercept_
array([25321.58301178])
```

```
import pickle
pickle.dump(model,open('SalaryPred.model','wb'))

model=pickle.load(open('SalaryPred.model','rb'))

yr_of_exp=float(input("Enter Years of Experience: "))
yr_of_exp_NP=np.array([[yr_of_exp]])
Salary=model.predict(yr_of_exp_NP)
Enter Years of Experience:  44
print("Estimated Salary for {} years of experience is {}: "
.format(yr_of_exp,Salary))

Estimated Salary for 44.0 years of experience is [[439969.45722514]]:
```

Name: S SHUVANTH Register Number: 250701708 eans Clustering

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline

df=pd.read_csv('Mall_Customers.csv')

df.info()
```

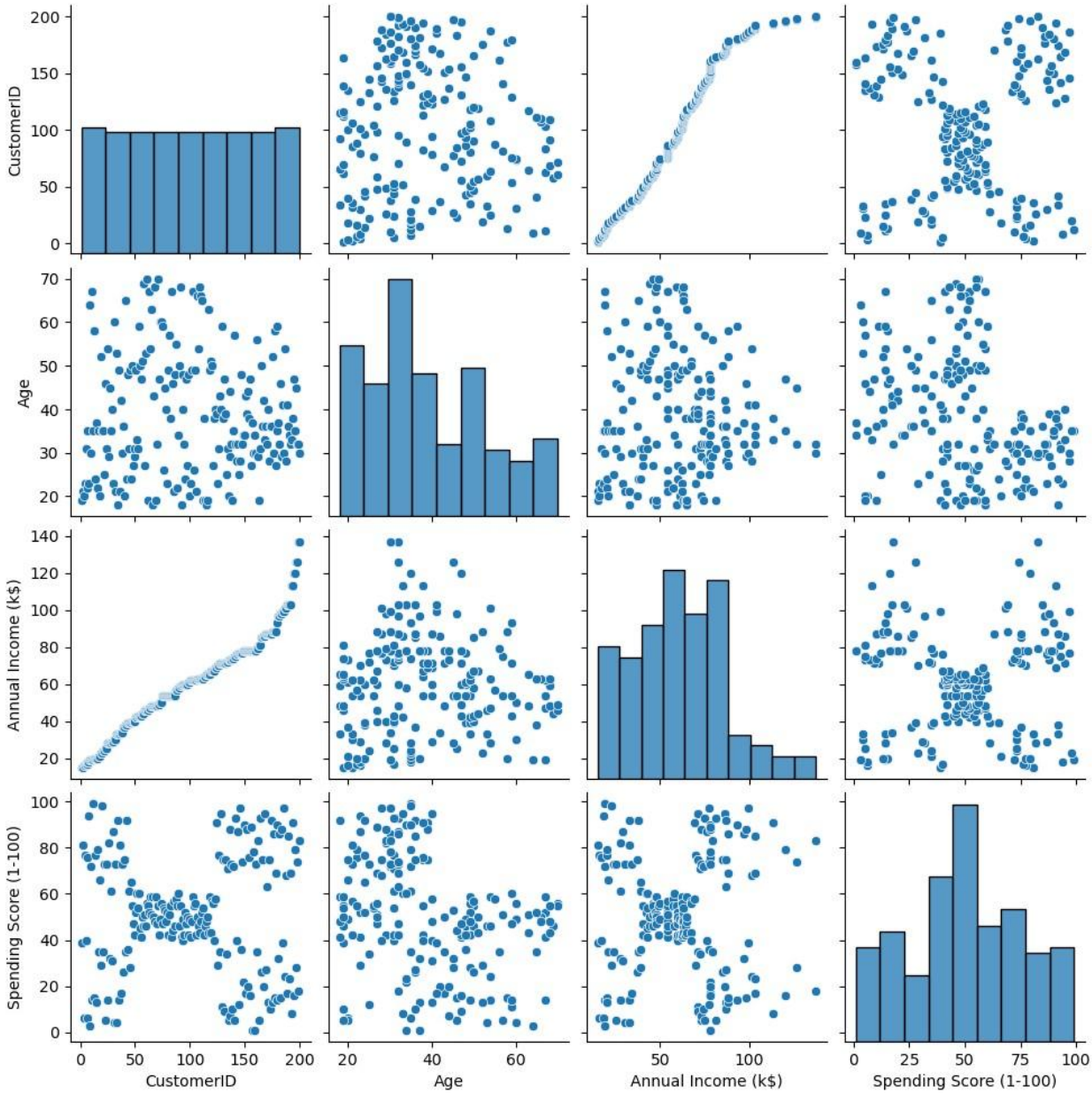
```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 5 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
0    CustomerID              200 non-null    int64
1    Gender                  200 non-null    object
2    Age                     200 non-null    int64
3    Annual Income (k$)      200 non-null    int64  4   Spending
Score (1-100)  200 non-null    int64
dtypes: int64(4), object(1)
memory usage: 7.9+ KB
```

```
df.head()
```

	CustomerID	Gender	Age	Annual Income (k$)	Spending Score (1-100)
0	1	Male	19	15	39
1	2	Male	21	15	81
2	3	Female	20	16	6
3	4	Female	23	16	77
4	5	Female	31	17	40

```
sns.pairplot(df)
```

```
<seaborn.axisgrid.PairGrid at 0x1f67ce53230>
```




```
features=df.iloc[:,[3,4]].values

from sklearn.cluster import KMeans
model=KMeans(n_clusters=5)
model.fit(features)
KMeans(n_clusters=5)

C:\Users\swath\anaconda3\Lib\site-packages\sklearn\cluster\
_kmeans.py:1419: UserWarning: KMeans is known to have a memory leak on
Windows with MKL, when there are less chunks than available threads.
You can avoid it by setting the environment variable
```

```
OMP_NUM_THREADS=1.
warnings.warn(
KMeans(n_clusters=5)
```

```
Final=df.iloc[:,[3,4]]
Final['label']=model.predict(features)
Final.head()
```

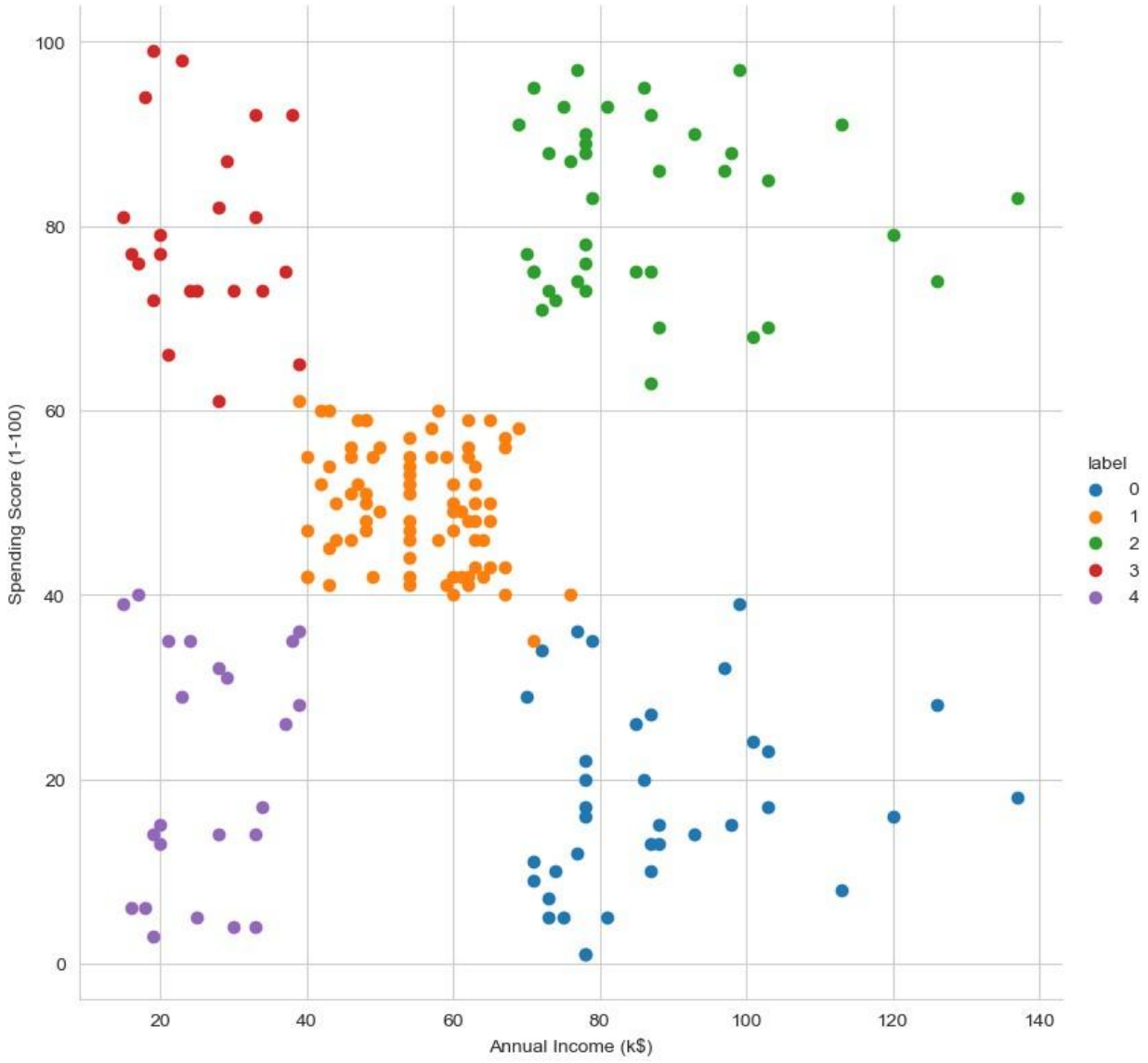
```
C:\Users\swath\AppData\Local\Temp\ipykernel_5828\470183701.py:2:
SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation:
https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#
returning-a-view-versus-a-copy
Final['label']=model.predict(features)
```

	Annual Income (k$)	Spending Score (1-100)	label
0	15	39	4
1	15	81	3
2	16	6	4
3	16	77	3
	17	40	4

4

```
sns.set_style("whitegrid")
sns.FacetGrid(Final,hue="label",height=8) \
.map(plt.scatter,"Annual Income (k$)", "Spending Score (1-100)") \
.add_legend();
plt.show()
```




```
features_el=df.iloc[:,[2,3,4]].values
from sklearn.cluster import KMeans
wcss=[]
for i in range(1,10):
    model=KMeans(n_clusters=i)
model.fit(features_el)
wcss.append(model.inertia_)
plt.plot(range(1,10),wcss)
```

```
  Cell In[11], line 5
    model=KMeans(n_clusters=i)
    ^
IndentationError: expected an indented block after 'for' statement on
line 4
```

Name: S SHUVANTH Register Number: 250701708

```
import numpy as np
import pandas as pd

df=pd.read_csv('Iris.csv')
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
0    sepal.length  150 non-null    float64
1    sepal.width   150 non-null    float64
2    petal.length  150 non-null    float64
3    petal.width   150 non-null    float64 4   variety       150 non-
null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
```

```
df.variety.value_counts()
```

```
variety

Setosa        50
Versicolor    50
Virginica     50
Name: count, dtype: int64
```

```
df.head()
```

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa

```
features=df.iloc[:,:-1].values
label=df.iloc[:,4].values

from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier

xtrain,xtest,ytrain,ytest=train_test_split(features,label,test_size=.2
,random_state=42)
model_KNN=KNeighborsClassifier(n_neighbors=5)
model_KNN.fit(xtrain,ytrain)

KNeighborsClassifier()
```

```
print(model_KNN.score(xtrain,ytrain))
print(model_KNN.score(xtest,ytest))
0.9666666666666667
1.0

from sklearn.metrics import confusion_matrix
confusion_matrix(label,model_KNN.predict(features))

array([[50,  0,  0],
[ 0, 47,  3],
       [ 0,  1, 49]])
```

Name: S SHUVANTH Register Number: 250701708

Experiment Name: Z test, T test, F Test, Anova Test T - Test

A sample of 10 students scored the following marks in an exam: [72, 68, 75, 70, 74, 69, 71, 73, 70, 72]
We want to test whether the average mark = 70 (μ0 = 70) at 5% significance level using python

```
import numpy as np from
scipy import stats
marks = np.array([72, 68, 75, 70, 74, 69, 71, 73, 70, 72])
mu_0 = 70
t_stat, p_value = stats.ttest_1samp(marks, mu_0) print(f"T-
statistic: {t_stat:.3f}") print(f"P-value: {p_value:.4f}") alpha =
0.05 if p_value < alpha:     print("Reject Null Hypothesis → Mean
is significantly different from 70.") else:     print("Fail to
Reject Null Hypothesis → No significant difference.")
T-statistic: 1.993
P-value: 0.0774

Fail to Reject Null Hypothesis → No significant difference.
```

Z - Test

```
Z-statistic: 2.400
P-value: 0.0164

Reject Null Hypothesis → Mean is significantly different from 50 g.
```

Anova Test

```
import numpy as np from math
import sqrt from scipy.stats
import norm x_bar = 51.2
mu_0 = 50 sigma = 3 n = 36
z_stat = (x_bar - mu_0) / (sigma / sqrt(n)) p_value = 2 * (1 -
norm.cdf(abs(z_stat))) print(f"Z-statistic: {z_stat:.3f}")
print(f"P-value: {p_value:.4f}") alpha = 0.05 if p_value < alpha:
print("Reject Null Hypothesis → Mean is significantly different
from 50 g.") else:     print("Fail to Reject Null Hypothesis → No
significant difference.")
```

Three fertilizers (A, B, C) were tested on crop yield (in kg). Is there a significant difference among fertilizers? (Use α = 0.05)

Fertilizer Yields

A 20, 22, 23

B 19, 20, 18

C 25, 27, 26

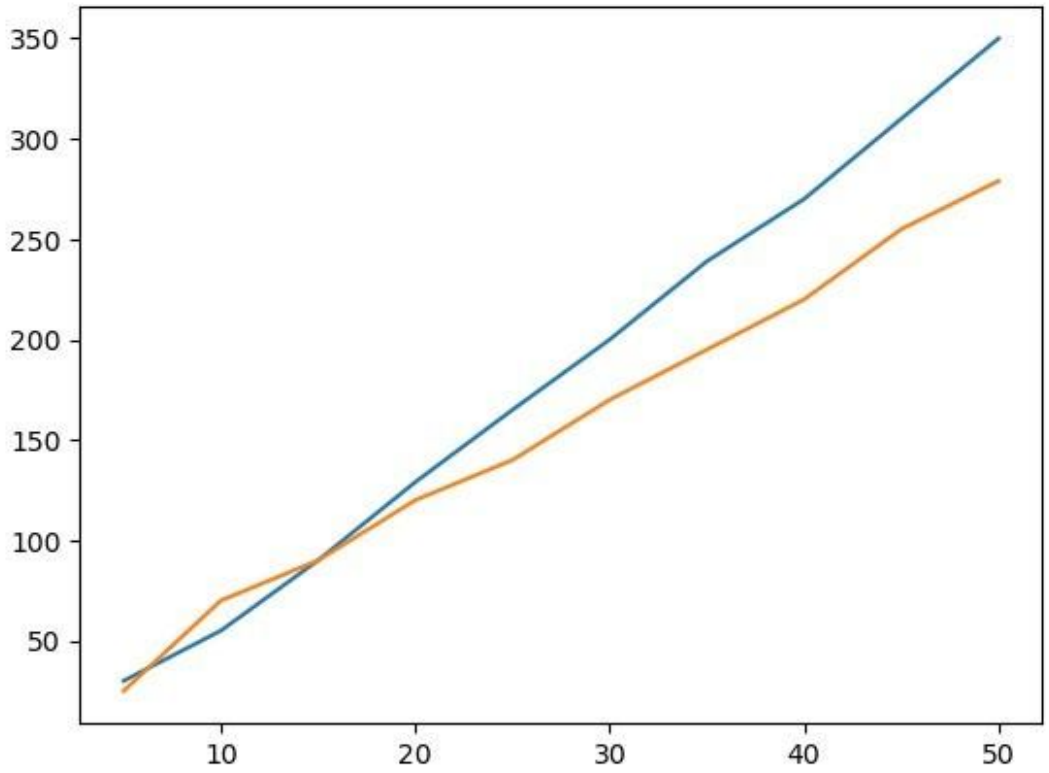
```
import numpy as np from
scipy import stats
A=[20,22,23]
B=[19,20,18]
C=[25,27,26]
f_stat,p_value=stats.f_oneway(A,B,C) print(f"F-statistic:
{f_stat:.3f}") print(f"P-value: {p_value:.4f}") alpha=0.05 if
p_value<alpha:     print("Reject Null Hypothesis-Means are
significantly different.") else:     print("Fail to Reject Null Hypo
```

Name: S SHUVANTH Register Number: 250701708

Experiment Name:Conduct an experiment to show data visualization using line plot,bar chart,pie chart

a.Conduct an experiment to show data visualization using line plot

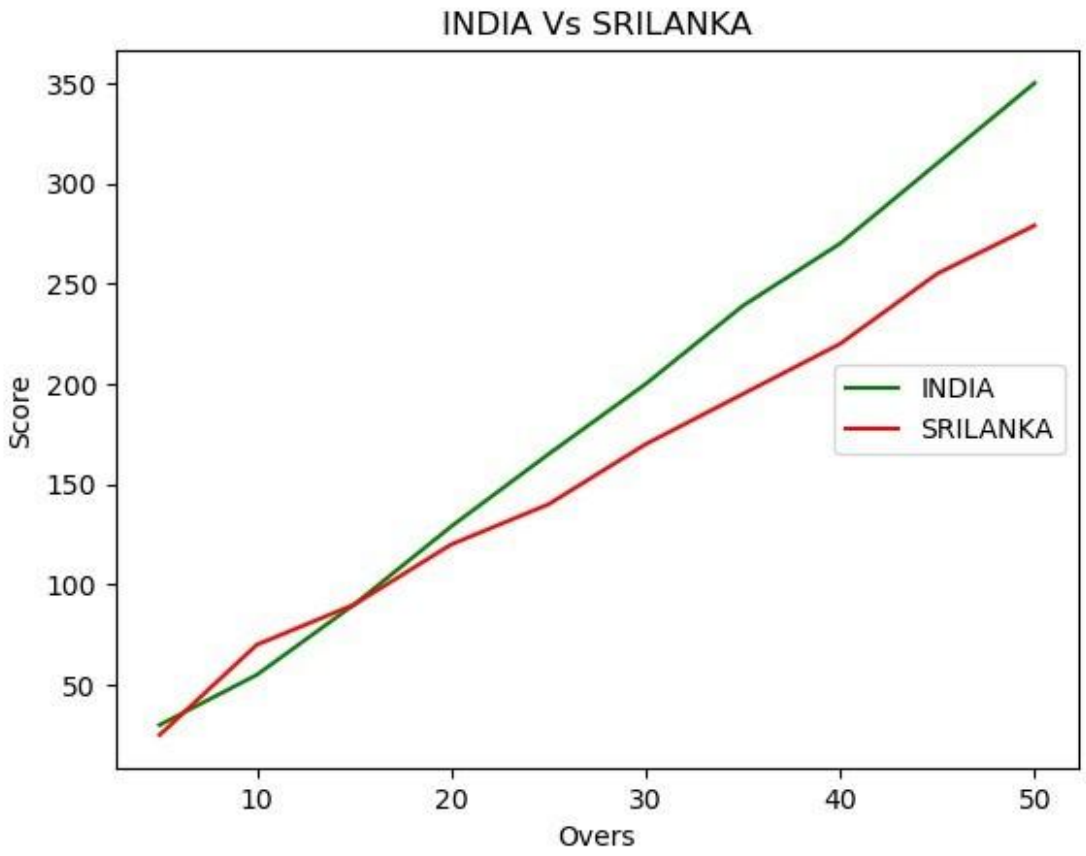
```
import matplotlib.pyplot as cricket
Overs=list(range(5,51,5))
Indian_Score=[30,55,90,129,165,200,239,270,310,350]
Srilankan_Score=[25,70,90,120,140,170,195,220,255,279]
cricket.plot(Overs,Indian_Score)
cricket.plot(Overs,Srilankan_Score)
cricket.show()
cricket.title("INDIA Vs SRILANKA")
cricket.xlabel("Overs")
cricket.ylabel("Score")
cricket.legend()
cricket.plot(Overs,Indian_Score,color="green",label="INDIA")
cricket.plot(Overs,Srilankan_Score,color="red",label="SRILANKA")
cricket.legend(loc="center right")
```




```
C:\Users\REC\AppData\Local\Temp\ipykernel_12032\506286342.py:11:
UserWarning: No artists with labels found to put in legend.  Note that
artists whose label start with an underscore are ignored when legend()
```

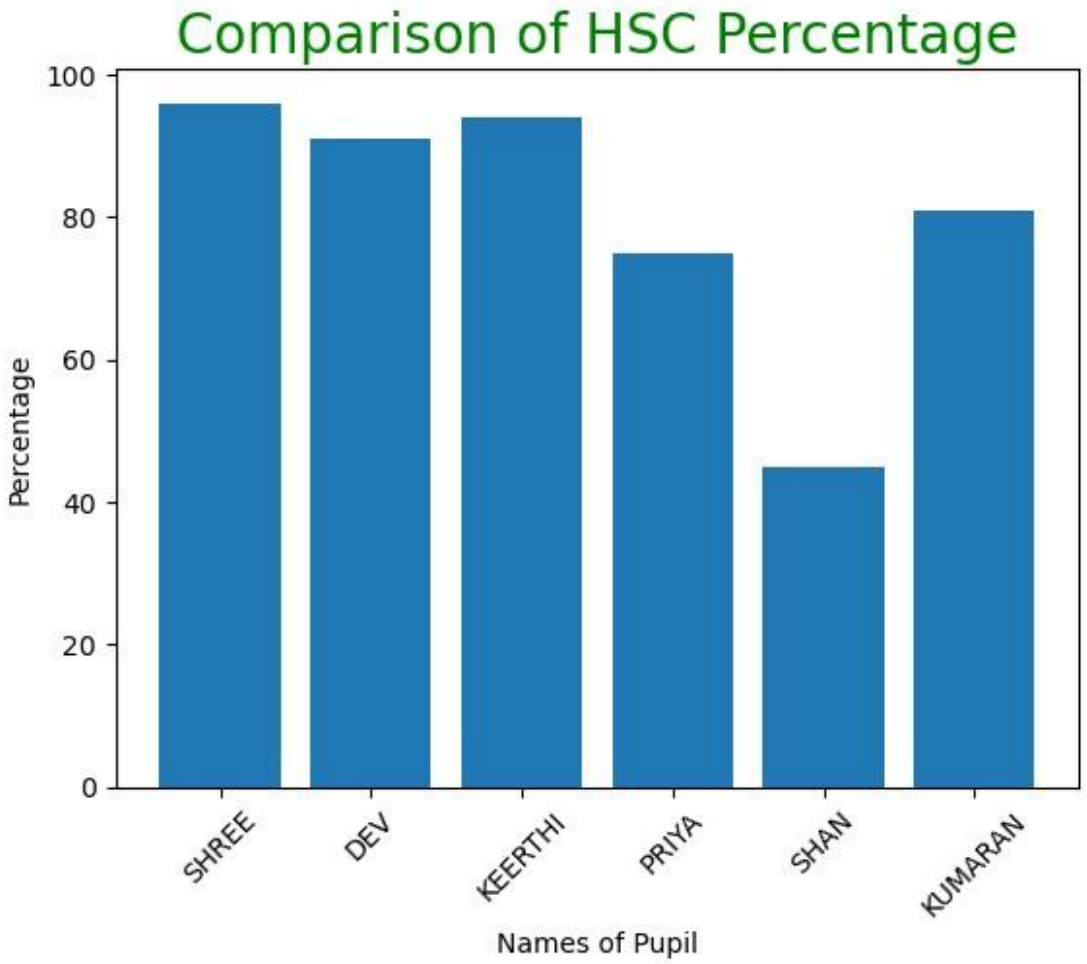
```
is called with no argument.
  cricket.legend()

<matplotlib.legend.Legend at 0x1659703fc50>
```



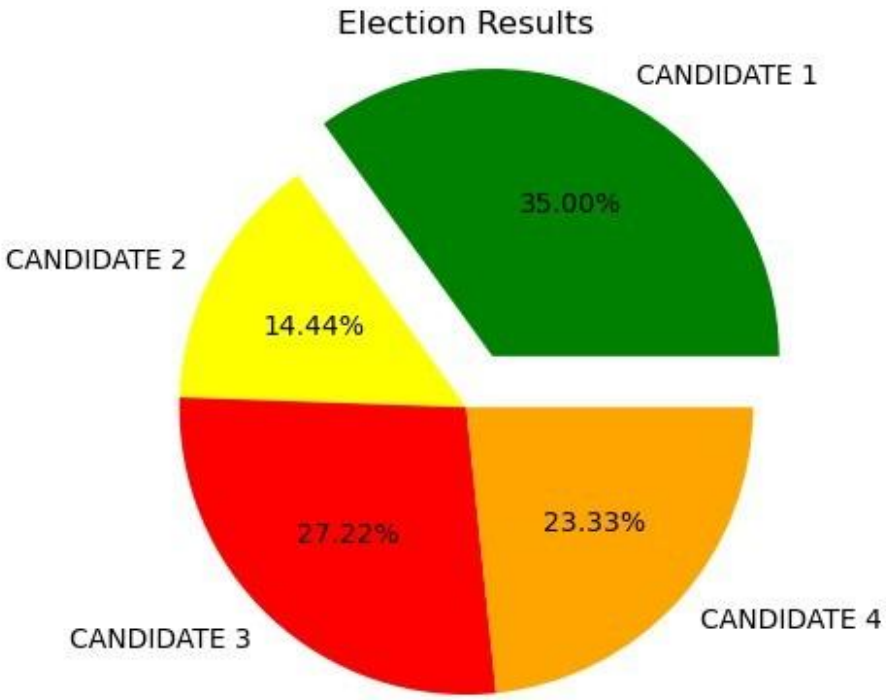
b.Conduct an experiment to show data visualization using bar chart

```
import matplotlib.pyplot as hscmark
import numpy as np
Names = ['SHREE', 'DEV', 'KEERTHI', 'PRIYA', 'SHAN', 'KUMARAN']
xaxis = np.arange(len(Names))
Percentage_hsc = [96, 91, 94, 75, 45, 81]
hscmark.bar(Names, Percentage_hsc)
hscmark.xticks(xaxis, Names, rotation=45)
hscmark.xlabel("Names of Pupil")
hscmark.ylabel("Percentage")
hscmark.title("Comparison of HSC Percentage", fontsize=20,
color="green") hscmark.show()
```



c.Conduct an experiment to show data visualization using pie chart

```
import matplotlib.pyplot as election
# Election data
labels = ['CANDIDATE 1', 'CANDIDATE 2', 'CANDIDATE 3', 'CANDIDATE 4']
Votes = [315, 130, 245, 210]
colors = ['green', 'yellow', 'red', 'orange']
explode = (0.2, 0, 0, 0) # Plotting the pie
chart
election.pie(Votes, labels=labels, colors=colors, explode=explode,
autopct='%0.2f%%')
election.title('Election Results')
election.show()
```


Election Results
CANDIDATE 1
35.00%
CANDIDATE 2
14.44%
27.22%
23.33%
CANDIDATE 3
CANDIDATE 4

Name: S SHUVANTH Register Number: 250701708

Experiment Name:To Count the frequency of occurrence of a word in a body of text is often needed during text processing.

```
import nltk
from nltk.tokenize import word_tokenize
from nltk.corpus import gutenberg
# Download the gutenberg corpus if not already installed
nltk.download('gutenberg')
nltk.download('punkt') # Ensure the punkt tokenizer models are also
downloaded
# Load the text from the gutenberg corpus
sample = gutenberg.raw("austen-emma.txt") # Change to an existing text
from the corpus
# Tokenize the sample text
token = word_tokenize(sample)
# Create a list of the first 50 tokens
wlist = [] for i in range(50):
wlist.append(token[i])
# Calculate the frequency of each word in the list
wordfreq = [wlist.count(w) for w in wlist]
# Print the word-frequency pairs
print("Pairs\n" + str(list(zip(wlist, wordfreq))))
```

```
Pairs
[('[', 1), ('Emma', 2), ('by', 1), ('Jane', 1), ('Austen', 1), ('1816',
1), (']', 1), ('VOLUME', 1), ('I', 2), ('CHAPTER', 1), ('I', 2), ('Emma
', 2), ('Woodhouse', 1), (',', 5), ('handsome', 1), (',', 5),
('clever'
, 1), (',', 5), ('and', 3), ('rich', 1), (',', 5), ('with', 2), ('a',
1
), ('comfortable', 1), ('home', 1), ('and', 3), ('happy', 1),
('disposi
tion', 1), (',', 5), ('seemed', 1), ('to', 1), ('unite', 1), ('some',
1
), ('of', 2), ('the', 2), ('best', 1), ('blessings', 1), ('of', 2),
('e
xistence', 1), (';', 1), ('and', 3), ('had', 1), ('lived', 1),
('nearly
', 1), ('twenty-one', 1), ('years', 1), ('in', 1), ('the', 2),
('world'
, 1), ('with', 2)]
```

In [1]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Create a sample diabetes dataset
data = {
    'Pregnancies': [6, 1, 8, 1, 0, 5, 3, 10, 2, 4],
    'Glucose': [148, 85, 183, 89, 137, 116, 78, 115, 197, 110],
    'BloodPressure': [72, 66, 64, 66, 40, 74, 50, 0, 70, 92],
    'SkinThickness': [35, 29, 0, 23, 35, 0, 32, 0, 45, 0],
    'Insulin': [0, 0, 0, 94, 168, 0, 88, 0, 543, 0],
    'BMI': [33.6, 26.6, 23.3, 28.1, 43.1, 25.6, 31.0, 35.3, 30.5, 37.6],
    'DiabetesPedigreeFunction': [0.627, 0.351, 0.672, 0.167, 2.288, 0.201, 0.248, 0
    'Age': [50, 31, 32, 21, 33, 30, 26, 29, 53, 30],
    'Outcome': [1, 0, 1, 0, 1, 0, 1, 0, 1, 0]
}

diabetes_df = pd.DataFrame(data)

# 1. Display the first five rows
print("First Five Rows:")
print(diabetes_df.head())

# 2. Display the last five rows
print("\nLast Five Rows:")
print(diabetes_df.tail())

# 3. Display dataset information
print("\nDataset Information:")
diabetes_df.info()

# 4. Display the number of rows and columns
print("\nDataset Shape:")
print(diabetes_df.shape)

# 5. Display column names
print("\nColumn Names:")
print(diabetes_df.columns)

# 6. Check for missing values
print("\nMissing Values:")
print(diabetes_df.isnull().sum())

# 7. Display statistical summary
print("\nStatistical Summary:")
print(diabetes_df.describe())

# 8. Check duplicate rows
print("\nNumber of Duplicate Rows:")
print(diabetes_df.duplicated().sum())

# 9. Visualize the correlation between columns
plt.figure(figsize=(10, 6))
```

```
sns.heatmap(diabetes_df.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title("Correlation Heatmap of Diabetes Dataset")
plt.show()

# 10. Visualize the outcome distribution
sns.countplot(data=diabetes_df, x='Outcome')
plt.title("Diabetes Outcome Distribution")
plt.xlabel("Outcome (0 = No Diabetes, 1 = Diabetes)")
plt.ylabel("Count")
plt.show()
```

```
First Five Rows:
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1

Last Five Rows:
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
5            5      116             74              0        0  25.6
6            3       78             50             32       88  31.0
7           10      115              0              0        0  35.3
8            2      197             70             45      543  30.5
9            4      110             92              0        0  37.6

   DiabetesPedigreeFunction  Age  Outcome
5                     0.201   30        0
6                     0.248   26        1
7                     0.134   29        0
8                     0.158   53        1
9                     0.191   30        0

Dataset Information:
<class 'pandas.DataFrame'>
RangeIndex: 10 entries, 0 to 9
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype
---  ------                    --------------  -----
 0   Pregnancies               10 non-null     int64
 1   Glucose                   10 non-null     int64
 2   BloodPressure             10 non-null     int64
 3   SkinThickness             10 non-null     int64
 4   Insulin                   10 non-null     int64
 5   BMI                       10 non-null     float64
 6   DiabetesPedigreeFunction  10 non-null     float64
 7   Age                       10 non-null     int64
 8   Outcome                   10 non-null     int64
dtypes: float64(2), int64(7)
memory usage: 852.0 bytes

Dataset Shape:
(10, 9)

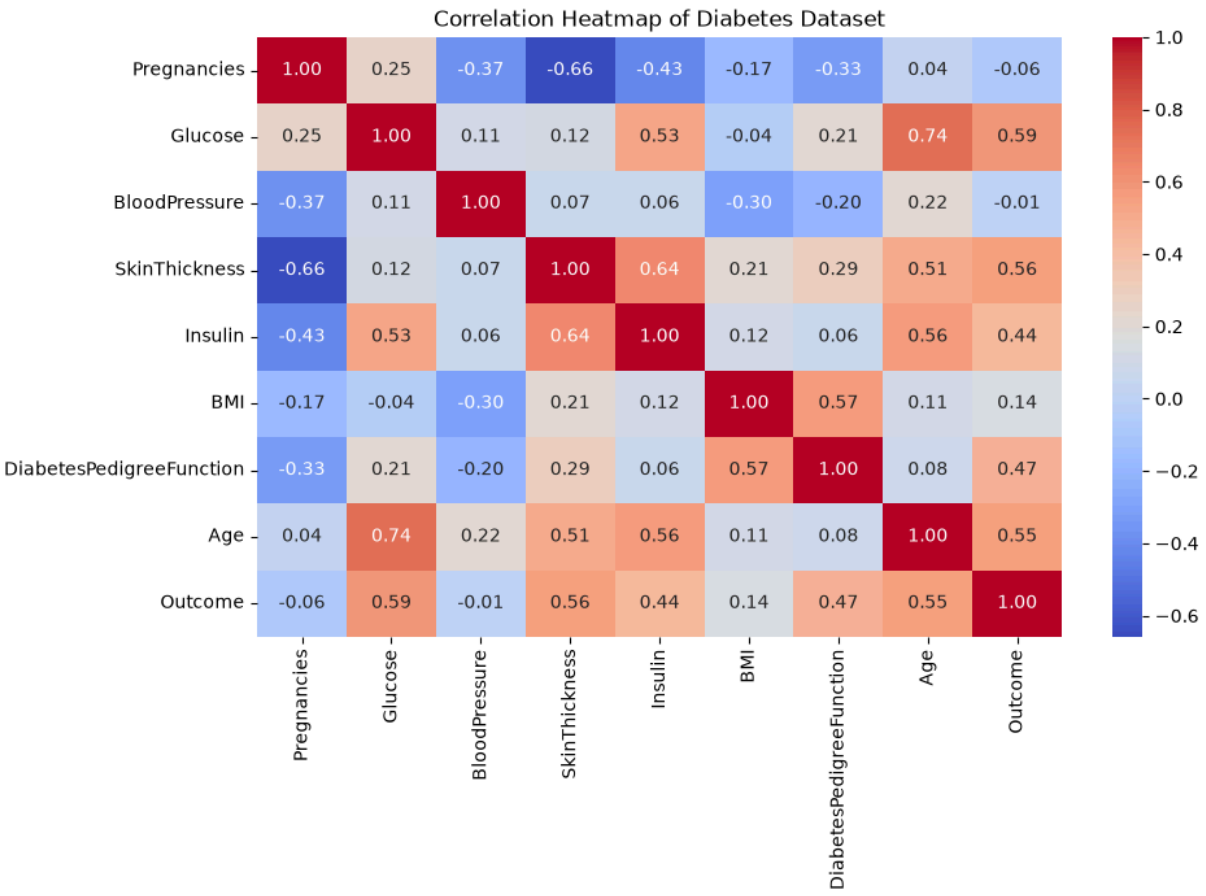
Column Names:
Index(['Pregnancies', 'Glucose', 'BloodPressure', 'SkinThickness', 'Insulin',
       'BMI', 'DiabetesPedigreeFunction', 'Age', 'Outcome'],
      dtype='str')
```

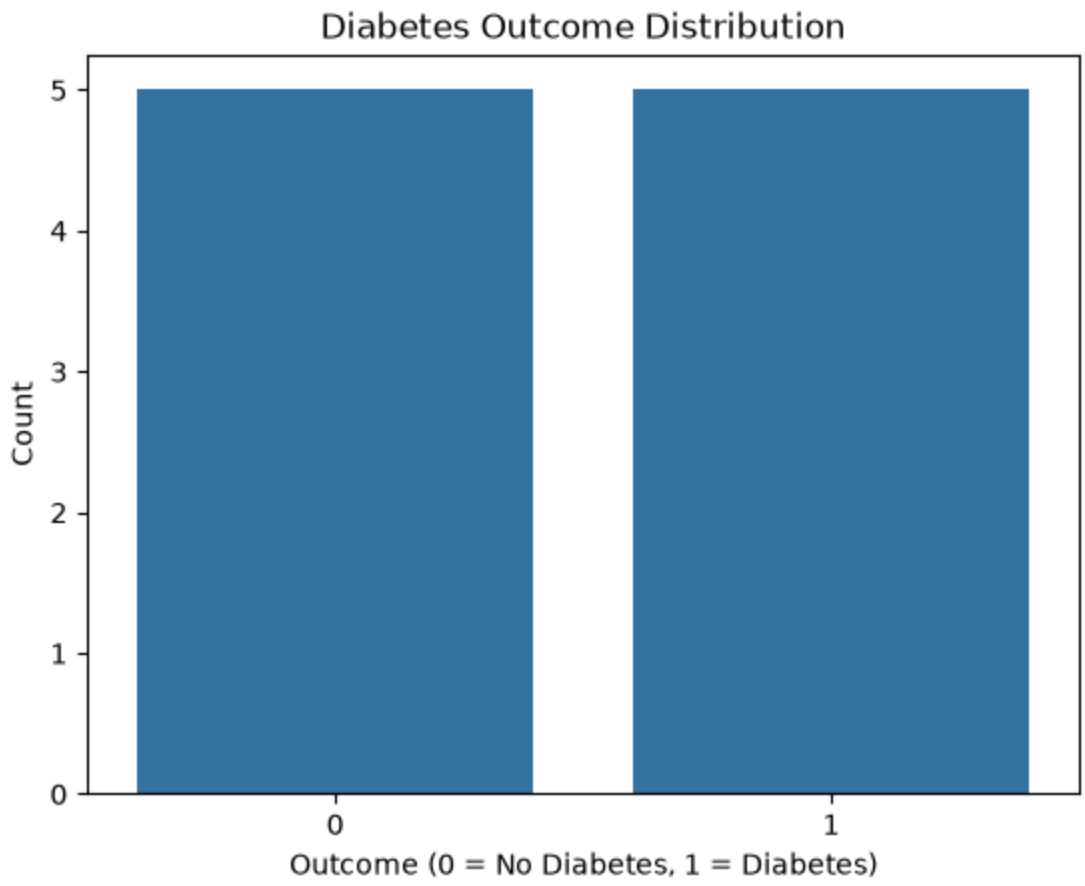
```
Missing Values:
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64

Statistical Summary:
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin  \
count    10.000000   10.000000      10.000000      10.000000   10.000000
mean      4.000000  125.800000      59.400000      19.900000   89.300000
std       3.265986   40.449694      25.051946      17.978073  169.937276
min       0.000000   78.000000       0.000000       0.000000    0.000000
25%       1.250000   94.250000      53.500000       0.000000    0.000000
50%       3.500000  115.500000      66.000000      26.000000    0.000000
75%       5.750000  145.250000      71.500000      34.250000   92.500000
max      10.000000  197.000000      92.000000      45.000000  543.000000

             BMI  DiabetesPedigreeFunction        Age    Outcome
count  10.000000                  10.00000  10.000000  10.000000
mean   31.470000                   0.50370  33.500000   0.500000
std     6.050537                   0.65616  10.102255   0.527046
min    23.300000                   0.13400  21.000000   0.000000
25%    26.975000                   0.17300  29.250000   0.000000
50%    30.750000                   0.22450  30.500000   0.500000
75%    34.875000                   0.55800  32.750000   1.000000
max    43.100000                   2.28800  53.000000   1.000000

Number of Duplicate Rows:
0
```


Correlation Heatmap of Diabetes Dataset
Pregnancies 1.00 0.25 -0.37 -0.66 -0.43 -0.17 -0.33 0.04 -0.06
Glucose 0.25 1.00 0.11 0.12 0.53 -0.04 0.21 0.74 0.59
BloodPressure -0.37 0.11 1.00 0.07 0.06 -0.30 -0.20 0.22 -0.01
SkinThickness -0.66 0.12 0.07 1.00 0.64 0.21 0.29 0.51 0.56
Insulin -0.43 0.53 0.06 0.64 1.00 0.12 0.06 0.56 0.44
BMI -0.17 -0.04 -0.30 0.21 0.12 1.00 0.57 0.11 0.14
DiabetesPedigreeFunction -0.33 0.21 -0.20 0.29 0.06 0.57 1.00 0.08 0.47
Age 0.04 0.74 0.22 0.51 0.56 0.11 0.08 1.00 0.55
Outcome -0.06 0.59 -0.01 0.56 0.44 0.14 0.47 0.55 1.00
Pregnancies
Glucose
BloodPressure
SkinThickness
Insulin
BMI
DiabetesPedigreeFunction
Age
Outcome
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
−0.4
−0.6


Diabetes Outcome Distribution
Count
0
1
2
3
4
5
0
1
Outcome (0 = No Diabetes, 1 = Diabetes)

In [ ]:

In [1]:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.tsa.seasonal import seasonal_decompose

# Create sample website traffic data
np.random.seed(42)

dates = pd.date_range(start='2025-01-01', periods=120, freq='D')
page_views = (
    500
    + np.arange(120) * 2
    + 50 * np.sin(2 * np.pi * np.arange(120) / 7)
    + np.random.normal(0, 20, 120)
)

# Add a few traffic spikes
page_views[40] += 200
page_views[85] += 250

data = pd.DataFrame({
    'Date': dates,
    'Page_Views': page_views
})

data.set_index('Date', inplace=True)

# Display the dataset
print("Website Traffic Data:")
print(data.head())

# 1. Time Series Decomposition
decomposition = seasonal_decompose(
    data['Page_Views'],
    model='additive',
    period=7
)

decomposition.plot()
plt.suptitle("Time Series Decomposition", y=1.02)
plt.tight_layout()
plt.show()

# 2. Moving Average
data['Page_Views_MA'] = data['Page_Views'].rolling(
    window=7
).mean()

data[['Page_Views', 'Page_Views_MA']].plot(
    figsize=(10, 5)
)
plt.title("Page Views with Moving Average")
plt.xlabel("Date")
plt.ylabel("Page Views")
```

```
plt.show()

# 3. Anomaly Detection using Z-score
mean = data['Page_Views'].mean()
std = data['Page_Views'].std()

data['Page_Views_Z'] = (
    data['Page_Views'] - mean
) / std

anomalies = data[data['Page_Views_Z'].abs() > 3]

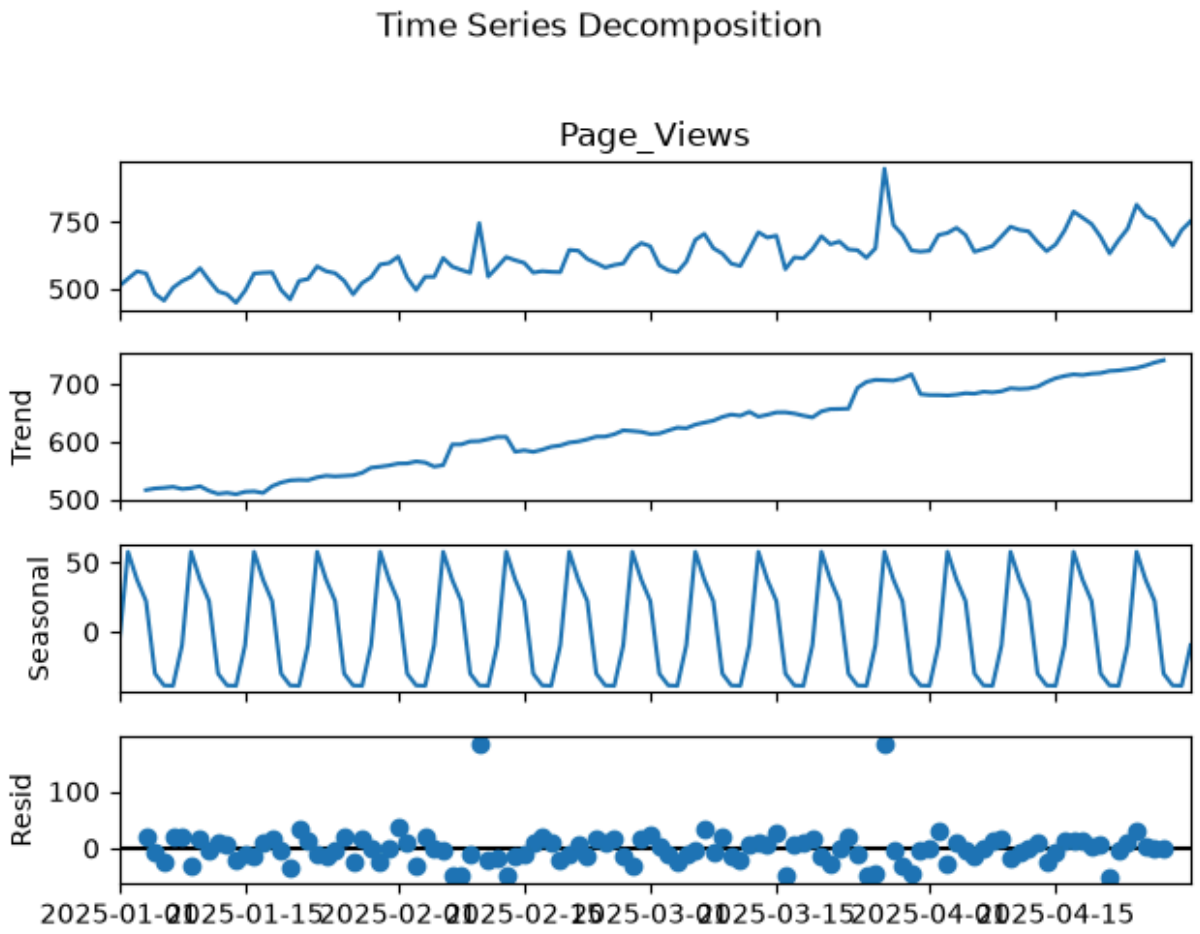
print("\nDetected Anomalies:")
print(anomalies[['Page_Views', 'Page_Views_Z']])

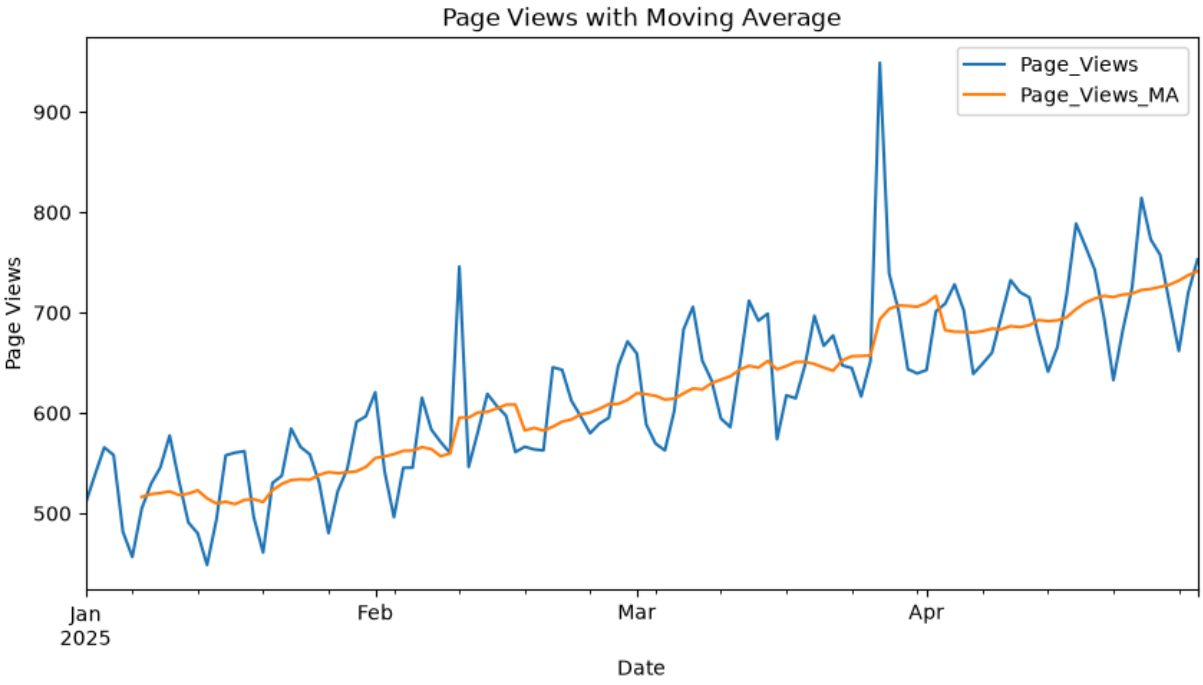
# Plot anomalies
plt.figure(figsize=(10, 5))
plt.plot(
    data.index,
    data['Page_Views'],
    label='Page Views'
)

plt.scatter(
    anomalies.index,
    anomalies['Page_Views'],
    color='red',
    label='Anomalies'
)

plt.title("Website Traffic Anomaly Detection")
plt.xlabel("Date")
plt.ylabel("Page Views")
plt.legend()
plt.show()
```

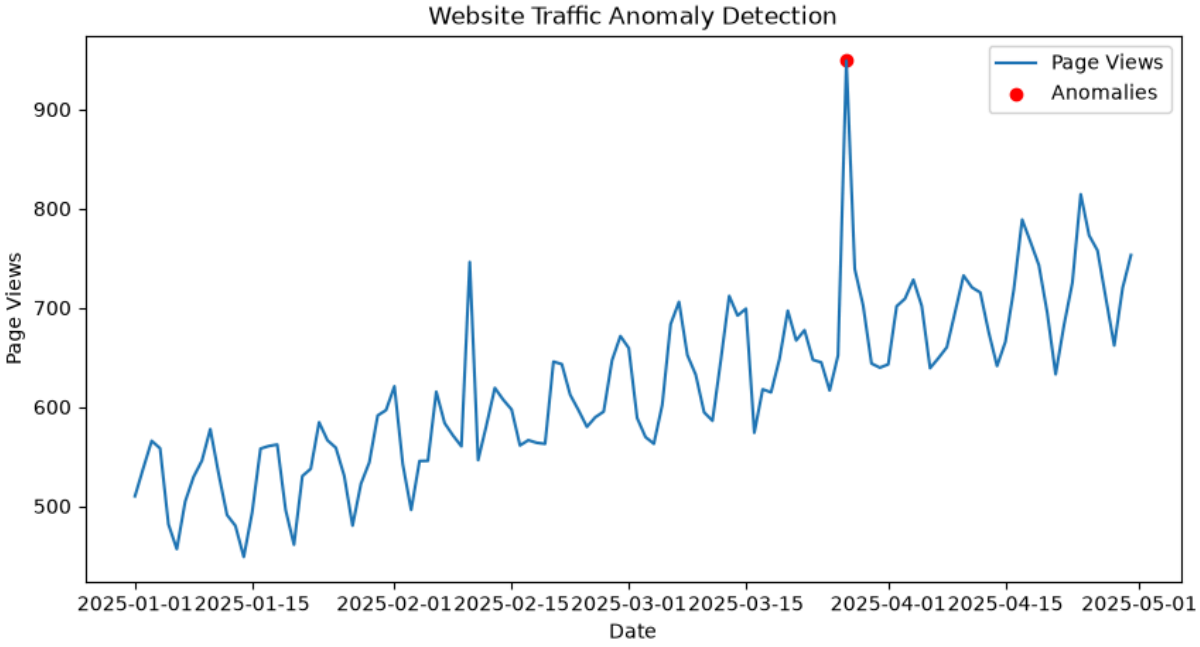
```
Website Traffic Data:
            Page_Views
Date
2025-01-01  509.934283
2025-01-02  538.326288
2025-01-03  565.700166
2025-01-04  558.154784
2025-01-05  481.622746
```





Detected Anomalies:

Date	Page_Views	Page_Views_Z
2025-03-27	949.056433	3.823627


Website Traffic Anomaly Detection
Page Views
Anomalies
900
800
700
600
500
Page Views
2025-01-01
2025-01-15
2025-02-01
2025-02-15
2025-03-01
2025-03-15
2025-04-01
2025-04-15
2025-05-01
Date

In [ ]:

Name: S SHUVANTH Register Number: 250701708
Experiment Name:Random Sampling and Sampling Distribution

```
import numpy as np
import matplotlib.pyplot as plt

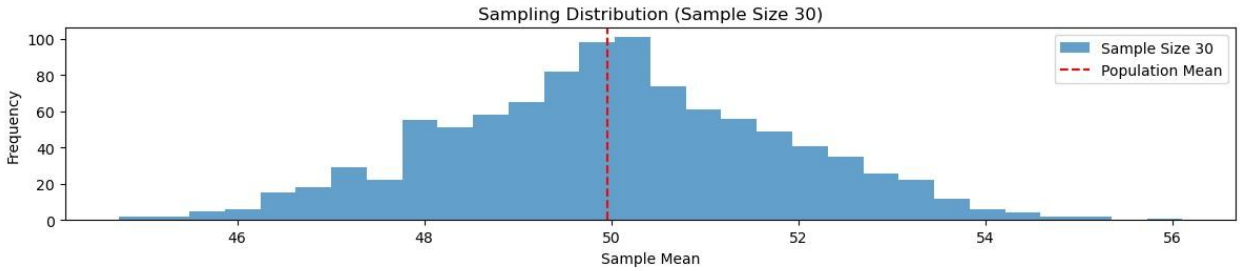
# Step 1: Generate a population (e.g., normal distribution)
population_mean = 50 population_std = 10 population_size =
100000
population = np.random.normal(population_mean, population_std,
population_size)

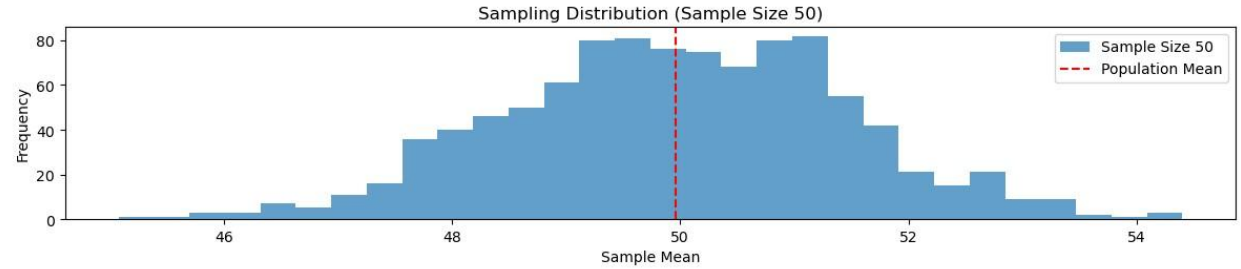
# Step 2: Random sampling
sample_sizes = [30, 50, 100] # different sample sizes to consider
num_samples = 1000 # number of samples for each sample size
sample_means = {}

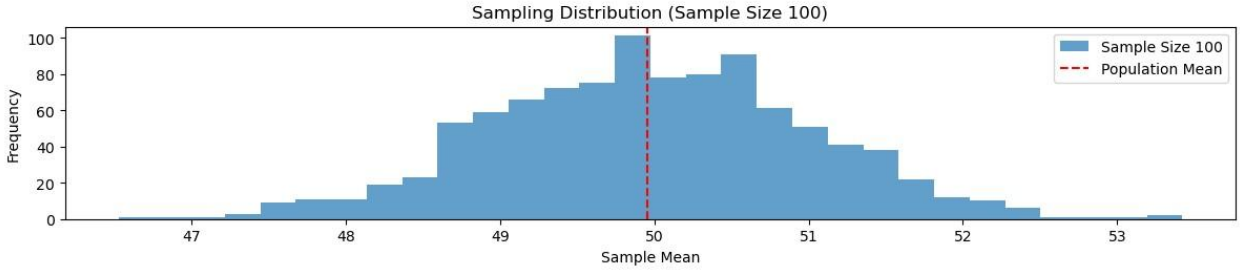
for size in sample_sizes:     sample_means[size] = []
for _ in range(num_samples):          sample =
np.random.choice(population, size=size, replace=False)
        sample_means[size].append(np.mean(sample))

# Step 3: Plotting sampling distributions
plt.figure(figsize=(12, 8))

for i, size in enumerate(sample_sizes):
plt.subplot(len(sample_sizes), 1, i+1)
    plt.hist(sample_means[size],  bins=30,  alpha=0.7,  label=f'Sample
Size {size}')
    plt.axvline(np.mean(population), color='red', linestyle='dashed',
linewidth=1.5, label='Population Mean')
    plt.title(f'Sampling Distribution (Sample Size {size})')
plt.xlabel('Sample Mean')     plt.ylabel('Frequency')
plt.legend() plt.tight_layout()
plt.show()
```


Sampling Distribution (Sample Size 30)
Sample Size 30
Population Mean
Frequency
Sample Mean


Sampling Distribution (Sample Size 50)
Sample Size 50
Population Mean
Frequency
Sample Mean


Sampling Distribution (Sample Size 100)
Sample Size 100
Population Mean
Frequency
Sample Mean

In [1]:
```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
%matplotlib inline
```

In [3]:
```
data=pd.read_csv('Iris.csv')
data
```

Out[3]:

	sepal.length	sepal.width	petal.length	petal.width	variety
0	5.1	3.5	1.4	0.2	Setosa
1	4.9	3.0	1.4	0.2	Setosa
2	4.7	3.2	1.3	0.2	Setosa
3	4.6	3.1	1.5	0.2	Setosa
4	5.0	3.6	1.4	0.2	Setosa
...	...	...	...	...	...
145	6.7	3.0	5.2	2.3	Virginica
146	6.3	2.5	5.0	1.9	Virginica
147	6.5	3.0	5.2	2.0	Virginica
148	6.2	3.4	5.4	2.3	Virginica
149	5.9	3.0	5.1	1.8	Virginica

150 rows × 5 columns

In [4]:
```
data.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column        Non-Null Count  Dtype  
---  ------        --------------  -----  
 0   sepal.length  150 non-null    float64
 1   sepal.width   150 non-null    float64
 2   petal.length  150 non-null    float64
 3   petal.width   150 non-null    float64
 4   variety       150 non-null    str    
dtypes: float64(4), str(1)
memory usage: 7.2 KB
```

In [5]:
```
data.describe()
```

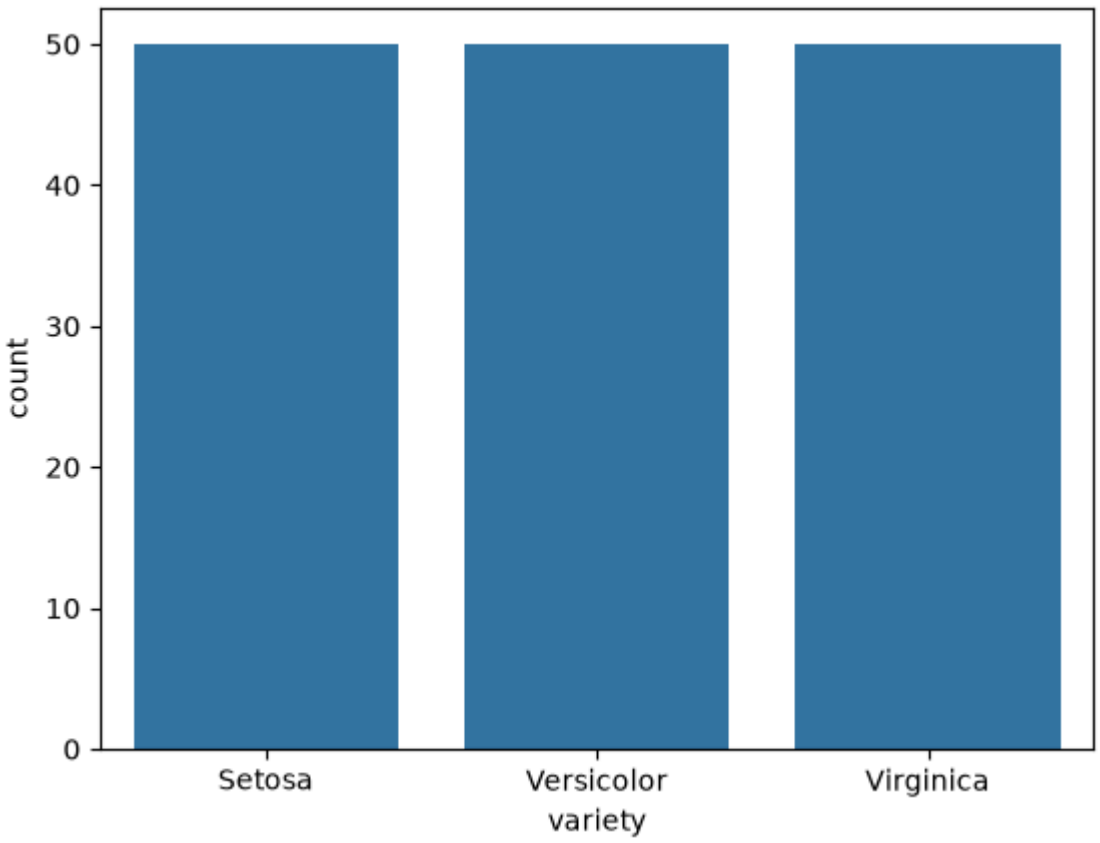
Out[5]:

	sepal.length	sepal.width	petal.length	petal.width
count	150.000000	150.000000	150.000000	150.000000
mean	5.843333	3.057333	3.758000	1.199333
std	0.828066	0.435866	1.765298	0.762238
min	4.300000	2.000000	1.000000	0.100000
25%	5.100000	2.800000	1.600000	0.300000
50%	5.800000	3.000000	4.350000	1.300000
75%	6.400000	3.300000	5.100000	1.800000
max	7.900000	4.400000	6.900000	2.500000

In [6]:
```
data.value_counts('variety')
```

Out[6]:
```
variety
Setosa        50
Versicolor    50
Virginica     50
Name: count, dtype: int64
```

In [7]:
```
sns.countplot(x='variety',data=data,)
plt.show()
```



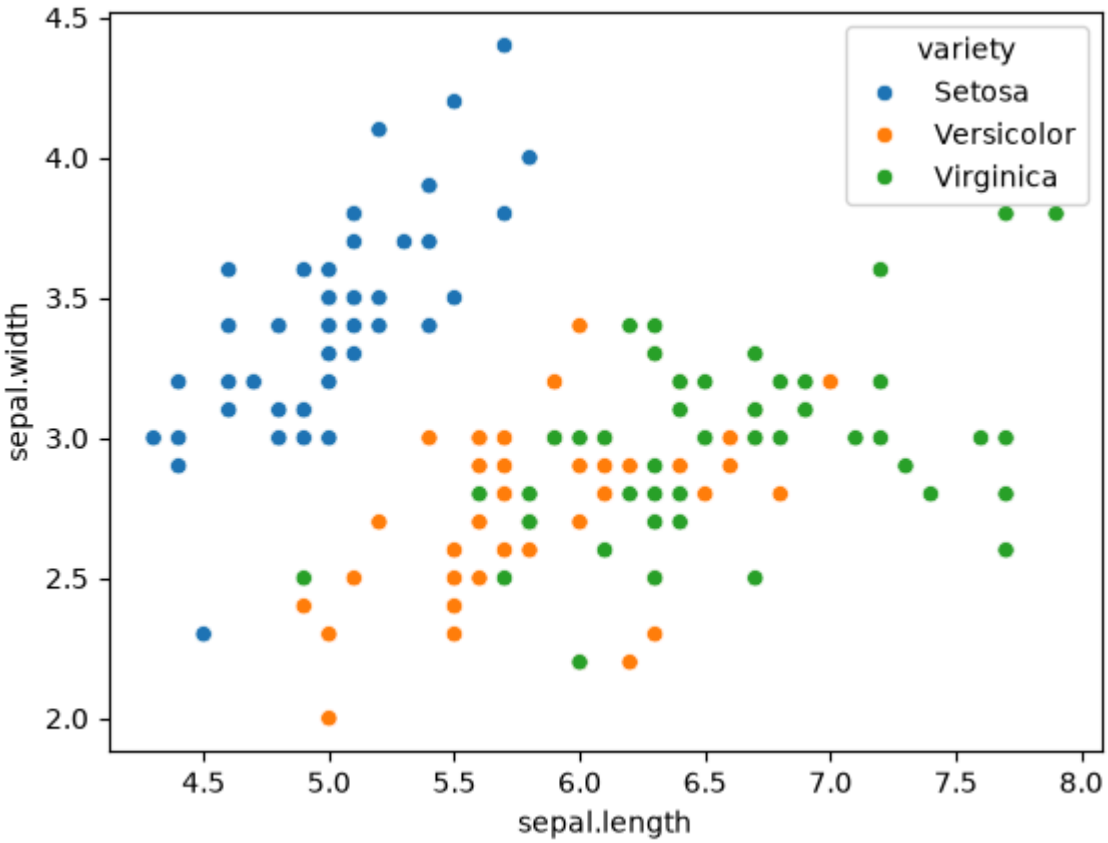
In [8]:
```
dummies=pd.get_dummies(data.variety)
FinalDataset=pd.concat([pd.get_dummies(data.variety),data.iloc[:,[0,1,2,3]]],axi
FinalDataset.head()
```

Out[8]:

	Setosa	Versicolor	Virginica	sepal.length	sepal.width	petal.length	petal.width
0	True	False	False	5.1	3.5	1.4	0.2
1	True	False	False	4.9	3.0	1.4	0.2
2	True	False	False	4.7	3.2	1.3	0.2
3	True	False	False	4.6	3.1	1.5	0.2
4	True	False	False	5.0	3.6	1.4	0.2

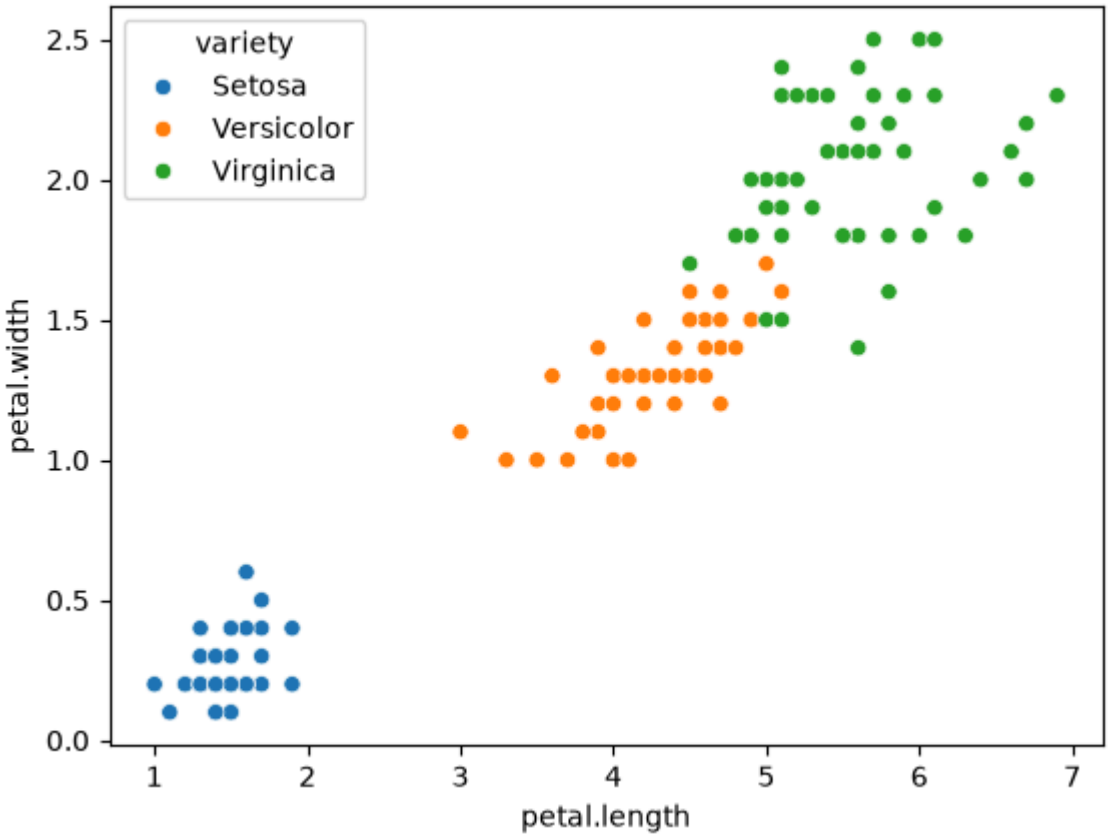
In [11]:
```
sns.scatterplot(x='sepal.length',y='sepal.width',hue='variety',data=data)
```

Out[11]:
```
<Axes: xlabel='sepal.length', ylabel='sepal.width'>
```

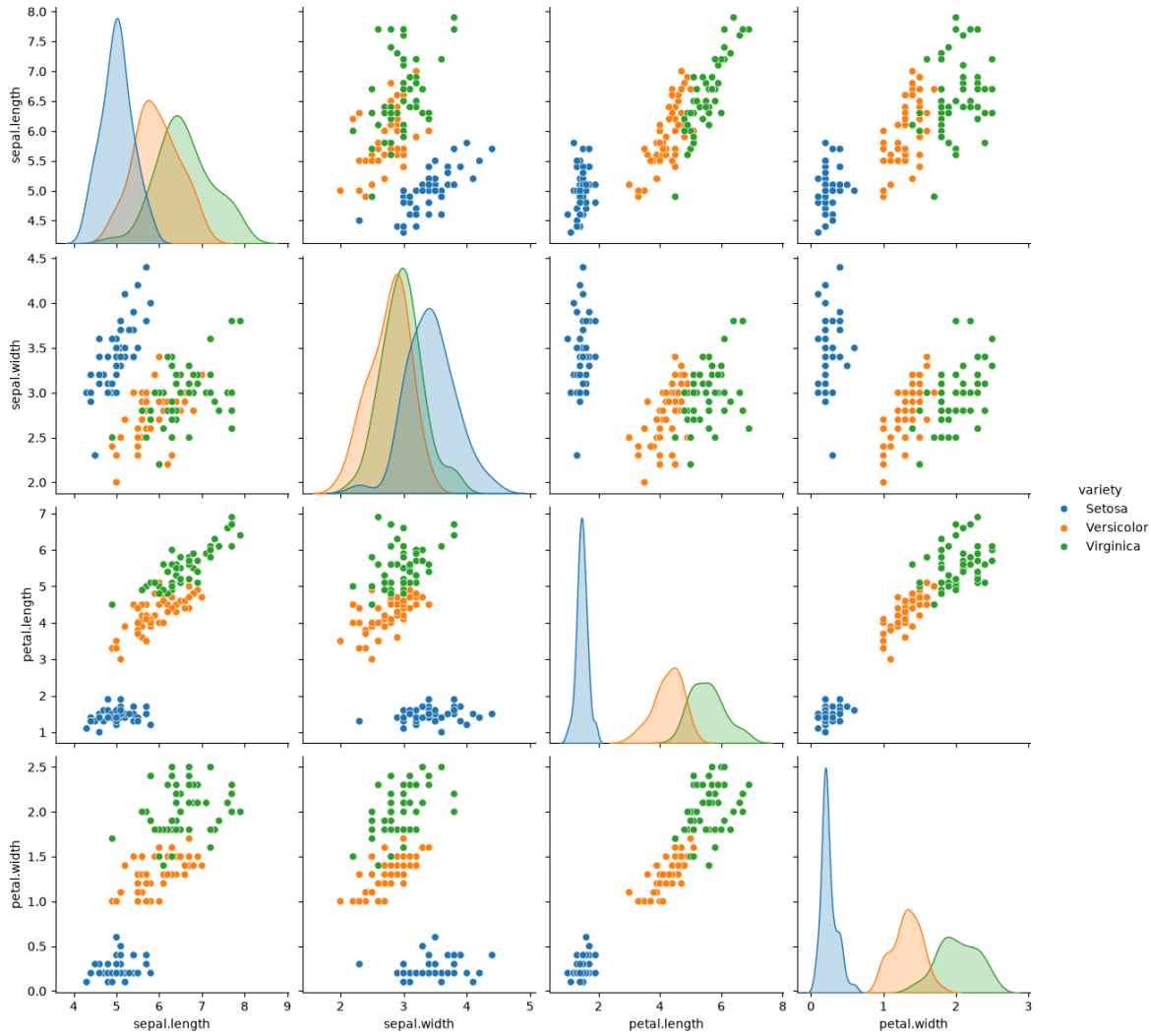


In [13]:
```
sns.scatterplot(x='petal.length',y='petal.width',hue='variety',data=data)
```

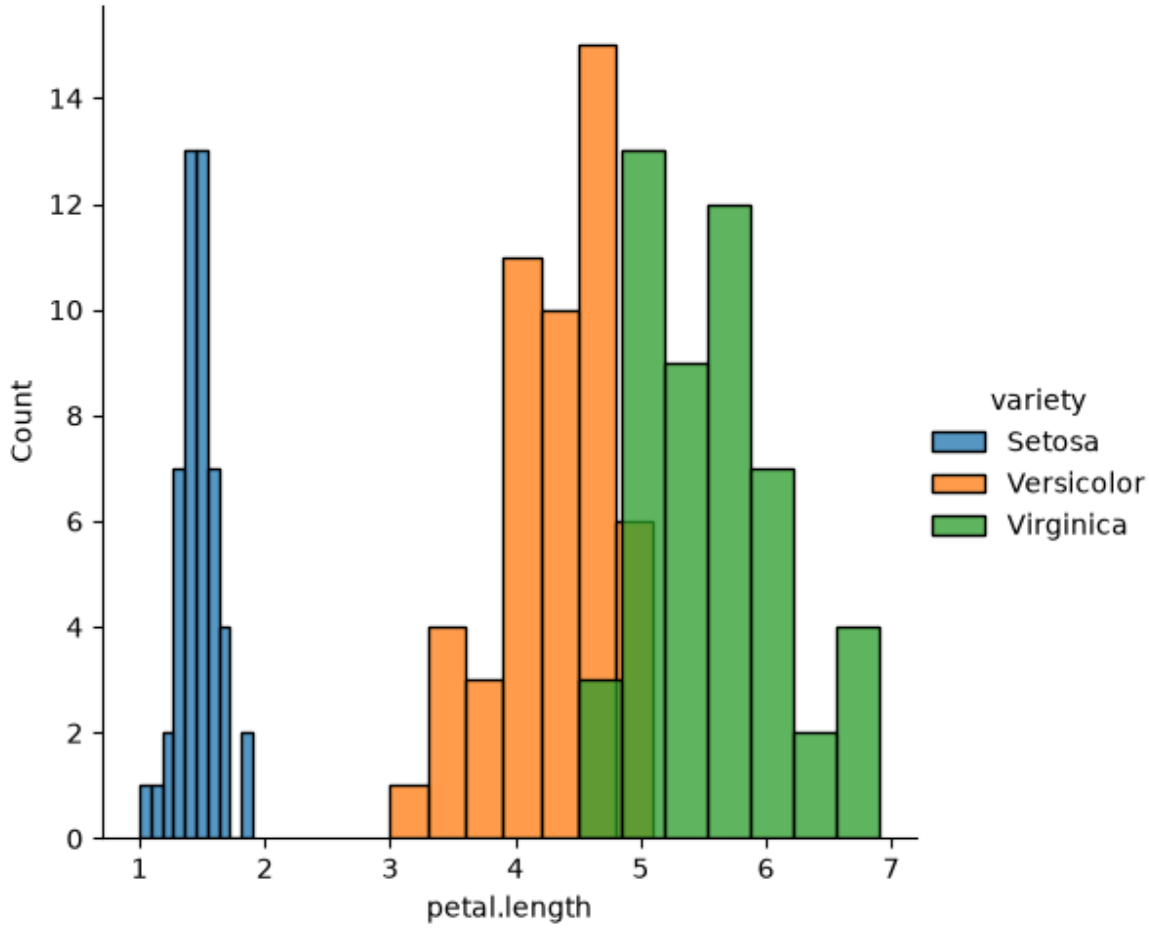
Out[13]:
```
<Axes: xlabel='petal.length', ylabel='petal.width'>
```



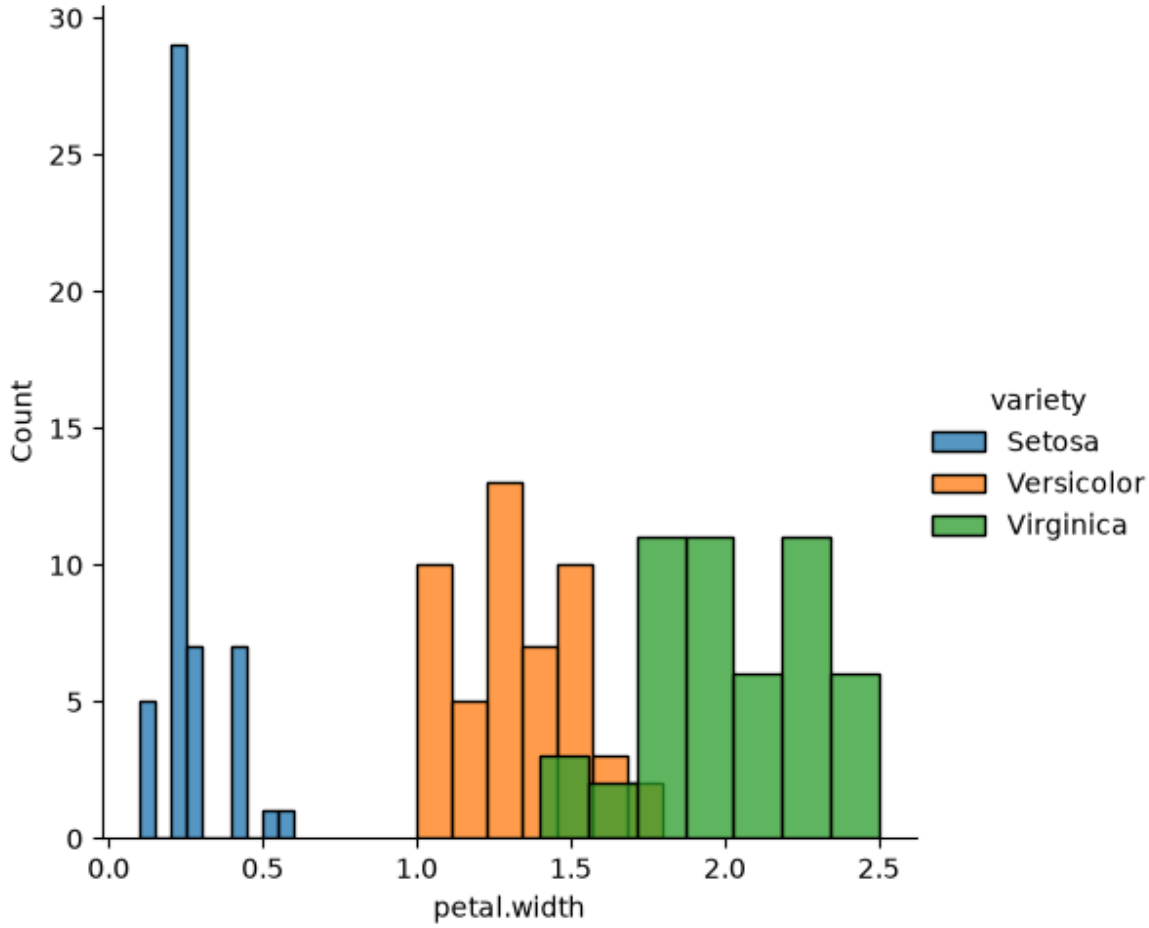
In [14]:
```
sns.pairplot(data,hue='variety',height=3);
```



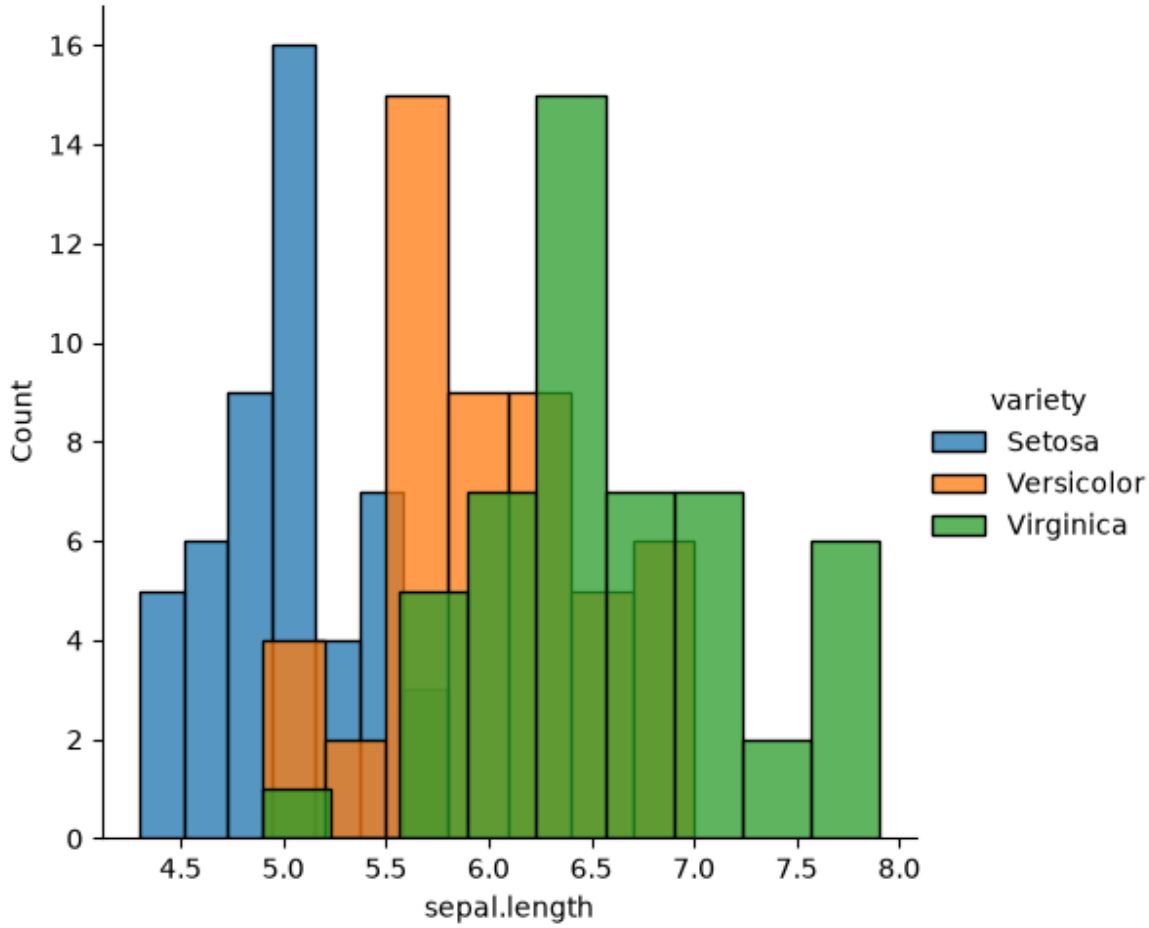
In [17]:
```
sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'petal.length').add_
```



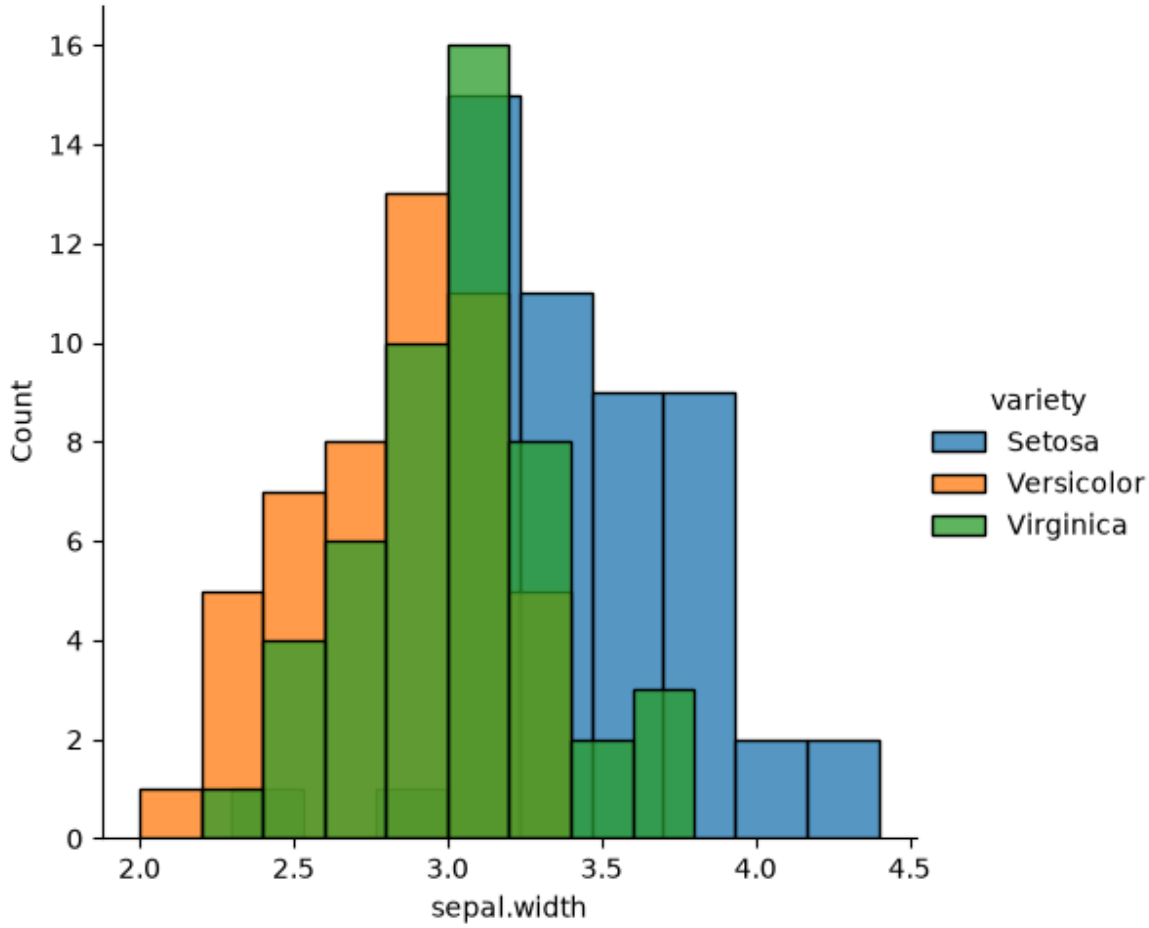
In [19]:
```
sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'petal.width').add_l
plt.show();
```



In [20]:
```
sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'sepal.length').add_
plt.show();
```



In [21]:
```
sns.FacetGrid(data,hue='variety',height=5).map(sns.histplot,'sepal.width').add_l
plt.show();
```


16
14
12
10
8
6
4
2
0
Count
2.0
2.5
3.0
3.5
4.0
4.5
sepal.width
variety
Setosa
Versicolor
Virginica

In [ ]:

Name: S SHUVANTH Register Number: 250701708

Experiment Name:Experiment to understand array function in Data science.

```
import numpy as np
array=np.random.randint(1,100,9)
array
array([ 6, 11, 49, 88, 98, 80, 24, 43, 12], dtype=int32)
np.sqrt(array)
array([2.44948974, 3.31662479, 7.        , 9.38083152, 9.89949494,
       8.94427191, 4.89897949, 6.55743852, 3.46410162])
array.ndim
```

```
1
```

```
new_array=array.reshape(3,3)
new_array
array([[ 6, 11, 49],
       [88, 98, 80],
       [24, 43, 12]], dtype=int32)
new_array.ndim
```

```
2
```

```
new_array.ravel()
array([ 6, 11, 49, 88, 98, 80, 24, 43, 12], dtype=int32)
newm=new_array.reshape(3,3)
newm
array([[ 6, 11, 49],
       [88, 98, 80],
       [24, 43, 12]], dtype=int32)
newm[2,1:3]
array([43, 12], dtype=int32)
newm[1:2,1:3]
array([[98, 80]], dtype=int32)
new_array[0:3,0:0]
```

```
array([], shape=(3, 0), dtype=int32)
```

```
new_array[0:2,0:1]
array([[ 6],

       [88]], dtype=int32)
new_array[0:3,0:1]
array([[ 6],
       [88],
       [24]], dtype=int32)
new_array[1:3]
array([[88, 98, 80],

       [24, 43, 12]], dtype=int32)
```

In [1]:

```python
import numpy as np
import pandas as pd

# 1. Create a DataFrame from a list
data = [
    [1, 'Smith', 50000],
    [2, 'Jones', 60000]
]

df = pd.DataFrame(data)
df.columns = ['Empd', 'Name', 'Salary']

print("Employee DataFrame:")
print(df)

# 2. Display DataFrame information
print("\nDataFrame Information:")
df.info()

# 3. Create an employee dataset
employee_data = {
    'emp id': [1, 2, 3, 4, 5, 6, 7],
    'name': [
        'SREE VARSSINI K S',
        'SREEMATHI B',
        'SREYA G',
        'SREYASKARI MULLAPUDI',
        'SRI AKASH U G',
        'SRI HARSHAVARDHANAN R',
        'SRI HARSHAVARDHANAN R'
    ],
    'salary': [5000, 6000, 7000, 5000, 8000, 3000, 6000]
}

df = pd.DataFrame(employee_data)

# 4. Display the first and last five rows
print("\nFirst Five Rows:")
print(df.head())

print("\nLast Five Rows:")
print(df.tail())

# 5. Display DataFrame information
print("\nEmployee DataFrame Information:")
df.info()

# 6. Select the salary column
print("\nSalary Column:")
print(df['salary'])

# 7. Calculate statistical measures
print("\nMean Salary:", df['salary'].mean())
print("Median Salary:", df['salary'].median())
```

```python
print("Mode Salary:", df['salary'].mode().tolist())
print("Salary Variance:", df['salary'].var())
print("Salary Standard Deviation:", df['salary'].std())

# 8. Display numerical summary
print("\nStatistical Summary:")
print(df.describe())

# 9. Display summary of all columns
print("\nComplete Summary:")
print(df.describe(include='all'))

# 10. Display column names
print("\nColumn Names:")
print(df.columns)

# 11. Convert DataFrame into a NumPy array
emp_array = df.values
print("\nEmployee Array:")
print(emp_array)

# 12. Create a DataFrame from the NumPy array
employee_DF = pd.DataFrame(emp_array, columns=df.columns)

print("\nFinal Employee DataFrame:")
print(employee_DF)
```

```
Employee DataFrame:
   Empd   Name  Salary
0     1  Smith   50000
1     2  Jones   60000

DataFrame Information:
<class 'pandas.DataFrame'>
RangeIndex: 2 entries, 0 to 1
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Empd    2 non-null      int64
 1   Name    2 non-null      str
 2   Salary  2 non-null      int64
dtypes: int64(2), str(1)
memory usage: 190.0 bytes

First Five Rows:
   emp id                  name  salary
0       1     SREE VARSSINI K S    5000
1       2           SREEMATHI B    6000
2       3               SREYA G    7000
3       4  SREYASKARI MULLAPUDI    5000
4       5         SRI AKASH U G    8000

Last Five Rows:
   emp id                   name  salary
2       3                SREYA G    7000
3       4   SREYASKARI MULLAPUDI    5000
4       5          SRI AKASH U G    8000
5       6  SRI HARSHAVARDHANAN R    3000
6       7  SRI HARSHAVARDHANAN R    6000

Employee DataFrame Information:
<class 'pandas.DataFrame'>
RangeIndex: 7 entries, 0 to 6
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   emp id  7 non-null      int64
 1   name    7 non-null      str
 2   salary  7 non-null      int64
dtypes: int64(2), str(1)
memory usage: 410.0 bytes

Salary Column:
0    5000
1    6000
2    7000
3    5000
4    8000
5    3000
6    6000
Name: salary, dtype: int64

Mean Salary: 5714.285714285715
```

```
Median Salary: 6000.0
Mode Salary: [5000, 6000]
Salary Variance: 2571428.5714285714
Salary Standard Deviation: 1603.5674514745463

Statistical Summary:
         emp id       salary
count  7.000000     7.000000
mean   4.000000  5714.285714
std    2.160247  1603.567451
min    1.000000  3000.000000
25%    2.500000  5000.000000
50%    4.000000  6000.000000
75%    5.500000  6500.000000
max    7.000000  8000.000000

Complete Summary:
          emp id                    name       salary
count   7.000000                       7     7.000000
unique       NaN                       6          NaN
top          NaN  SRI HARSHAVARDHANAN R          NaN
freq         NaN                       2          NaN
mean    4.000000                     NaN  5714.285714
std     2.160247                     NaN  1603.567451
min     1.000000                     NaN  3000.000000
25%     2.500000                     NaN  5000.000000
50%     4.000000                     NaN  6000.000000
75%     5.500000                     NaN  6500.000000
max     7.000000                     NaN  8000.000000

Column Names:
Index(['emp id', 'name', 'salary'], dtype='str')

Employee Array:
[[1 'SREE VARSSINI K S' 5000]
 [2 'SREEMATHI B' 6000]
 [3 'SREYA G' 7000]
 [4 'SREYASKARI MULLAPUDI' 5000]
 [5 'SRI AKASH U G' 8000]
 [6 'SRI HARSHAVARDHANAN R' 3000]
 [7 'SRI HARSHAVARDHANAN R' 6000]]

Final Employee DataFrame:
  emp id                   name salary
0      1      SREE VARSSINI K S   5000
1      2            SREEMATHI B   6000
2      3                SREYA G   7000
3      4   SREYASKARI MULLAPUDI   5000
4      5          SRI AKASH U G   8000
5      6  SRI HARSHAVARDHANAN R   3000
6      7  SRI HARSHAVARDHANAN R   6000
```

In [ ]: